Experiment :1 Narmadha k 241801181

```
#EXERCISE 1 - DATA PREPROCESSING AND CLEANING
#NARMADHA K 241801181
import pandas as pd
# Load Titanic Dataset
df = pd.read_csv("Titanic.csv")
print("Titanic Dataset Loaded Successfully!")
```

```
Titanic Dataset Loaded Successfully!
```

```
#NARMADHA K 241801181
#Display the First Few Rows
print("First 5 Rows:")
print(df.head())
#Display Dataset Information
print("\nDataset Information:")
print(df.info())
#Check Missing Values
print("\nMissing Values:")
print(df.isnull().sum())
```

```
First 5 Rows:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name  Sex   Age  SibSp  Parch  \
0                            Braund, Mr. Owen Harris    1  22.0      1      0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...    0  38.0      1      0
2                             Heikkinen, Miss. Laina    0  26.0      0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)    0  35.0      1      0
4                           Allen, Mr. William Henry    1  35.0      0      0

             Ticket      Fare    Cabin Embarked
0         A/5 21171 -0.502445  Unknown        S
1          PC 17599  0.786845      C85        C
2  STON/O2. 3101282 -0.488854  Unknown        S
3            113803  0.420730     C123        S
4            373450 -0.486337  Unknown        S

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
```

```
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)    0  35.0      1      0
4                           Allen, Mr. William Henry    1  35.0      0      0
```

```
             Ticket      Fare    Cabin Embarked
0         A/5 21171 -0.502445  Unknown        S
1          PC 17599  0.786845      C85        C
2  STON/O2. 3101282 -0.488854  Unknown        S
3            113803  0.420730     C123        S
4            373450 -0.486337  Unknown        S

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    int64
 5   Age          891 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        891 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(6), object(4)
memory usage: 83.7+ KB
None

Missing Values:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
Embarked       2
dtype: int64
```

[32] ✓ 0s

```
#NARMADHA K 241801181
#Do Forward Fill and Backward Fill on 'Age'
df_forward = df.copy()
df_backward = df.copy()
# Forward Fill
df_forward["Age"] = df_forward["Age"].ffill()
# Backward Fill
df_backward["Age"] = df_backward["Age"].bfill()
# Apply Forward Fill to main dataset
df["Age"] = df["Age"].ffill()
print(df["Age"].head())
```

```
0    22.0
1    38.0
2    26.0
3    35.0
4    35.0
Name: Age, dtype: float64
```

[31] ✓ 0s

```
#NARMADHA K 241801181
#Fill Missing Values in 'Cabin' with 'Unknown' and Limit to 5
df["Cabin"] = df["Cabin"].fillna("Unknown")
print(df["Cabin"].head())
```

```
0    Unknown
1        C85
2    Unknown
3       C123
4    Unknown
Name: Cabin, dtype: object
```

[30] ✓ 0s

```
#NARMADHA K 241801181
#Remove Duplicate Records
print("Duplicate Records:", df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicates Removed Successfully!")
```

```
Duplicate Records: 0
Duplicates Removed Successfully!
```

[29] ✓ 0s

```
#NARMADHA K 241801181
#Encode the Categorical Column 'Sex' using LabelEncoder
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df["Sex"] = label_encoder.fit_transform(df["Sex"])
print(df["Sex"].head())
```

Run cell (Ctrl+Enter)
cell executed since last change
executed by NARMADHA K
11:05 AM (4 minutes ago)
executed in 0.003s

[29] ✓ 0s

```
#NARMADHA K 241801181
#Encode the Categorical Column 'Sex' using LabelEncoder
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
df["Sex"] = label_encoder.fit_transform(df["Sex"])
print(df["Sex"].head())
```

```
0    1
1    0
2    0
3    0
4    1
Name: Sex, dtype: int64
```

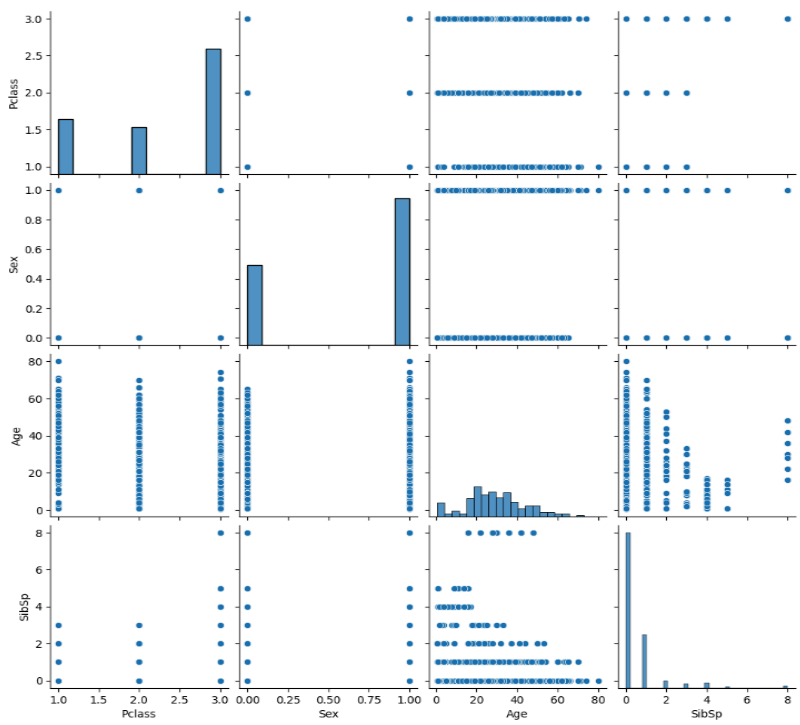
[28] ✓ 0s

```
#NARMADHA K 241801181
#Scale the Numerical Feature 'Fare' using StandardScaler
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df["Fare"] = scaler.fit_transform(df[["Fare"]])
print(df["Fare"].head())
```

```
0   -0.502445
1    0.786845
2   -0.488854
3    0.420730
4   -0.486337
Name: Fare, dtype: float64
```

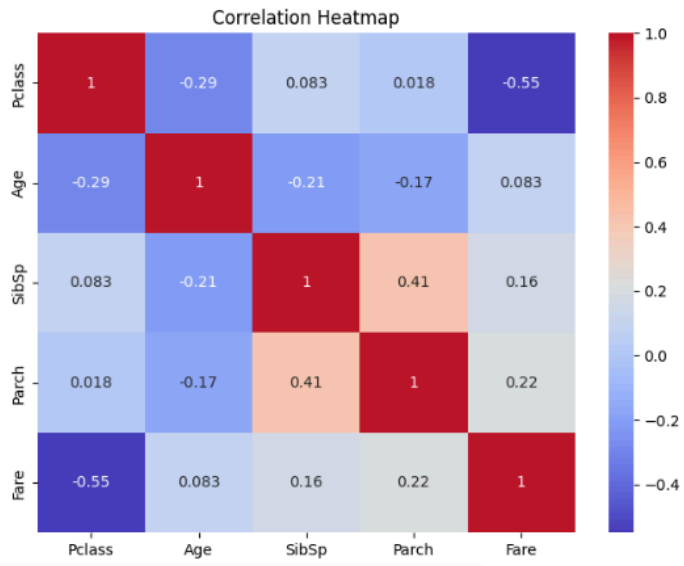
[27] ✓ 2s

```
#NARMADHA K 241801181
#Pair Plot of Selected Features ('Pclass', 'Sex', 'Age', 'SibSp')
import seaborn as sns
import matplotlib.pyplot as plt
sns.pairplot(df[["Pclass", "Sex", "Age", "SibSp"]])
plt.show()
```



[26]
✓ 0s

```
#NARMADHA K 241801181
#Display the Correlation Heatmap for ('Pclass', 'Age', 'SibSp', 'Parch', 'Fare')
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(8,6))
correlation = df[["Pclass", "Age", "SibSp", "Parch", "Fare"]].corr()
sns.heatmap(correlation,
            annot=True,
            cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```



exp no:2

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[5] 0s

```
# Narmadha k
# 241801181
#EX 2: DATA DISCOVERY AND EXPLORATORY DATA ANALYSIS
# TITANIC DATASET
# ============================================================
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
```

[6] 0s

```
# Narmadha k
# 241801181
# 1. Use titanic.csv and convert it into dataframe
# ------------------------------------------------------------

df = pd.read_csv("titanic.csv")

print("Titanic Dataset:")
print(df.head())
```

```
Titanic Dataset:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3
```

exp no:2

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

[7] 0s

```
# Narmadha k
# 241801181
# 2. Understand the data (shape, info, describe)
# ------------------------------------------------------------

print("\nShape of Dataset:")
print(df.shape)

print("\nDataset Information:")
print(df.info())

print("\nStatistical Description:")
print(df.describe())
```

```
Shape of Dataset:
(891, 12)

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None

Statistical Description:
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
```

[8]

```
# Narmadha k
# 241801181
# 3. Fill missing values in Age using SimpleImputer
# ------------------------------------------------------------

imputer = SimpleImputer(strategy="mean")

df["Age"] = imputer.fit_transform(df[["Age"]]).ravel()

print("\nMissing Age values after imputation:")
print(df["Age"].isnull().sum())
```

```
Missing Age values after imputation:
0
```

[9]

```
# Narmadha k
# 241801181
# 4. Fill missing Cabin with "Unknown"
# ------------------------------------------------------------

df["Cabin"] = df["Cabin"].fillna("Unknown")

print("\nMissing Cabin values:")
print(df["Cabin"].isnull().sum())
```

```
Missing Cabin values:
0
```

[10]

```
# Narmadha k
# 241801181
# 5. Fill missing Embarked with the mode
# ------------------------------------------------------------

df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])

print("\nMissing Embarked values:")
```

exp no:2

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Missing Embarked values:
0
```

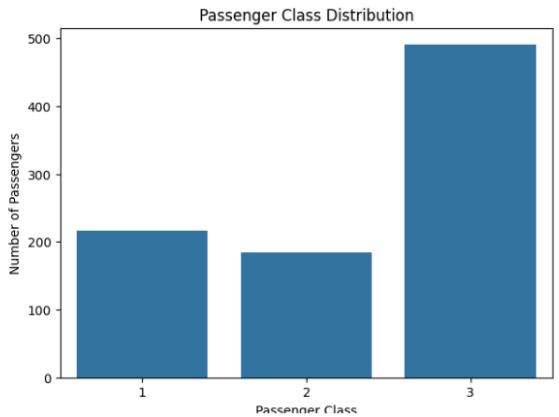
[11] ✓ 0s

```
# Narmadha k
# 241801181
# 6. Analyse Passenger Class using countplot
# ------------------------------------------------------------

plt.figure(figsize=(7, 5))

sns.countplot(x="Pclass", data=df)

plt.title("Passenger Class Distribution")
plt.xlabel("Passenger Class")
plt.ylabel("Number of Passengers")
plt.show()
```



[12] ✓ 0s

```
# Narmadha k
# 241801181
# 7. List names of passengers who are female and survived
# ------------------------------------------------------------

female_survived = df[
    (df["Sex"] == "female") &
    (df["Survived"] == 1)
]

print("\n7. Female Passengers Who Survived:")
print(female_survived["Name"].to_string(index=False))
```

```
                            Sagesser, Mlle. Emma
                           Baclini, Miss. Eugenie
                 Stanley, Miss. Amy Zillah Elsie
                              Doling, Miss. Elsie
                    O'Leary, Miss. Hanora "Norah"
 Taylor, Mrs. Elmer Zebley (Juliet Cummins Wright)
Brown, Mrs. Thomas William Solomon (Elizabeth C...
                          Turja, Miss. Anna Sofia
                Madill, Miss. Georgette Alexandra
                                Karun, Miss. Manca
                 Mullens, Miss. Katherine "Katie"
 Astor, Mrs. John Jacob (Madeleine Talmadge Force)
                    Kelly, Mrs. Florence "Fannie"
                             Cleaver, Miss. Alice
  Mayne, Mlle. Berthe Antonine ("Mrs de Villiers")
                    Endres, Miss. Caroline Louise
              Troutt, Miss. Edwina Celia "Winnie"
                Harper, Miss. Annie Jessie "Nina"
      Renouf, Mrs. Peter Henry (Lillian Jefferys)
                           Mannion, Miss. Margareth
                  Allen, Miss. Elisabeth Walton
            Ryerson, Miss. Susan Parker "Suzette"
                           Sinkkonen, Miss. Anna
                              Wells, Miss. Joan
                  Herman, Mrs. Samuel (Jane Laver)
Rothes, the Countess. of (Lucy Noel Martha Dyer...
         Carter, Mrs. William Ernest (Lucile Polk)
              Hogeboom, Mrs. John C (Anna Andrews)
             Hocking, Mrs. Elizabeth (Eliza Needs)
                      Emanuel, Miss. Virginia Ethel
Robert, Mrs. Edward Scott (Elisabeth Walton McM...
```

[13] ✓ 0s

```
# Narmadha k
# 241801181
# 8. Find all passengers who are in 3rd class
#    and under the age of 18
# ------------------------------------------------------------

third_class_under18 = df[
    (df["Pclass"] == 3) &
    (df["Age"] < 18)
]

print("\n8. 3rd Class Passengers Under Age 18:")
print(third_class_under18[["Name", "Age", "Pclass"]].to_string(index=False))
```

```
                                Calic, Mr. Jovo 17.00       3
                     Panula, Master. Eino Viljami  1.00       3
Goldsmith, Master. Frank John William "Frankie"  9.00       3
                             Rice, Master. Arthur  4.00       3
                     Johnson, Miss. Eleanor Ileen  1.00       3
           Asplund, Master. Clarence Gustaf Hugo  9.00       3
             Kink-Heilmann, Miss. Luise Gretchen  4.00       3
                      Strom, Miss. Telma Matilda  2.00       3
                        Carr, Miss. Helen "Ellen" 16.00       3
                Sunderland, Mr. Victor Francis 16.00       3
                   Asplund, Miss. Lillian Gertrud  5.00       3
              Asplund, Master. Edvin Rojj Felix  3.00       3
                      Panula, Mr. Ernesti Arvid 16.00       3
                             Rice, Master. Eric  7.00       3
                   de Pelsmaeker, Mr. Alfons 16.00       3
             Vander Planke, Mr. Leo Edmondus 16.00       3
          Coutts, Master. William Loch "William"  3.00       3
                               Elias, Mr. Tannous 15.00       3
                      Palsson, Miss. Stina Viola  3.00       3
                    Nakid, Miss. Maria ("Mary")  1.00       3
                 Goodwin, Master. Sidney Leonard  1.00       3
                      Van Impe, Miss. Catharina 10.00       3
                     Kallio, Mr. Nikolai Erland 17.00       3
                  Baclini, Miss. Marie Catherine  5.00       3
                   Baclini, Miss. Helene Barbara  0.75       3
                        Hirvonen, Miss. Hildur E  2.00       3
                Goodwin, Master. Harold Victor  9.00       3
           Coutts, Master. Eden Leslie "Neville"  9.00       3
                              Calic, Mr. Petar 17.00       3
                          Elias, Mr. Joseph Jr 17.00       3
```

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

Table of contents

[14]

```
adha k
01181
# 9. Find passengers who are in class 1
#    and older than 40
# ------------------------------------------------------------

class1_over40 = df[
    (df["Pclass"] == 1) &
    (df["Age"] > 40)
]

print("\n9. Class 1 Passengers Older Than 40:")
print(class1_over40[["Name", "Age", "Pclass"]].to_string(index=False))
```

```
                                        Stead, Mr. William Thomas 62.0       1
                                             Taussig, Mr. Emil 52.0       1
                         Graham, Mrs. William Thompson (Edith Junkins) 58.0       1
                                      Andrews, Miss. Kornelia Theodosia 63.0       1
                       Baxter, Mrs. James (Helene DeLaudeniere Chaput) 50.0       1
                                            Partner, Mr. Austen 45.5       1
                                  Burns, Miss. Elizabeth Margaret 41.0       1
                                     Blackwell, Mr. Stephen Weart 45.0       1
                   Warren, Mrs. Frank Manley (Anna Sophia Atkinson) 60.0       1
                                          Bidois, Miss. Rosalie 42.0       1
                                    Silvey, Mr. William Baird 50.0       1
                                        Fortune, Mr. Mark 64.0       1
                                  Peuchen, Major. Arthur Godfrey 52.0       1
                                       Goldenberg, Mr. Samuel L 49.0       1
                                     Millet, Mr. Francis Davis 65.0       1
                                           Anderson, Mr. Harry 48.0       1
                                             Gee, Mr. Arthur H 47.0       1
                                    Smart, Mr. John Montgomery 56.0       1
                                       Kent, Mr. Edward Austin 58.0       1
                                    Molson, Mr. Harry Markland 55.0       1
                                       Artagaveytia, Mr. Ramon 71.0       1
                                  Eustis, Miss. Elizabeth Mussey 54.0       1
                   Rothschild, Mrs. Martin (Elizabeth L. Barrett) 54.0       1
                                       Walker, Mr. William Anderson 47.0       1
                   Hippach, Mrs. Louis Albert (Ida Sophia Fischer) 44.0       1
                                 Butt, Major. Archibald Willingham 45.0       1
                                        Douglas, Mr. Walter Donald 50.0       1
                                     Nicholson, Mr. Arthur Ernest 64.0       1
                                              Wright, Mr. George 62.0       1
Duff Gordon, Lady. (Lucille Christiana Sutherland) ("Mrs Morgan") 48.0       1
                        Appleton, Mrs. Edward Dale (Charlotte Lamson) 53.0       1
```

```python
# Narmadha k
# 241801181
# 10. Find passengers survived in the above category
#     (Class 1 and Age > 40)
# ------------------------------------------------------------

class1_over40_survived = df[
    (df["Pclass"] == 1) &
    (df["Age"] > 40) &
    (df["Survived"] == 1)
]

print("\n10. Class 1 Passengers Older Than 40 Who Survived:")
print(class1_over40_survived[
    ["Name", "Age", "Pclass", "Survived"]
].to_string(index=False))
```

```
10. Class 1 Passengers Older Than 40 Who Survived:
                                                Name  Age  Pclass  Survived
                                  Bonnell, Miss. Elizabeth 58.0       1         1
                       Harper, Mrs. Henry Sleeper (Myna Haxtun) 49.0       1         1
                    Romaine, Mr. Charles Hallace ("Mr C Rolmane") 45.0       1         1
                        Brown, Mrs. James Joseph (Margaret Tobin) 44.0       1         1
                                  Lurette, Miss. Elise 58.0       1         1
                    Graham, Mrs. William Thompson (Edith Junkins) 58.0       1         1
                           Andrews, Miss. Kornelia Theodosia 63.0       1         1
                    Baxter, Mrs. James (Helene DeLaudeniere Chaput) 50.0       1         1
                           Burns, Miss. Elizabeth Margaret 41.0       1         1
                 Warren, Mrs. Frank Manley (Anna Sophia Atkinson) 60.0       1         1
                                  Bidois, Miss. Rosalie 42.0       1         1
                           Peuchen, Major. Arthur Godfrey 52.0       1         1
                                Goldenberg, Mr. Samuel L 49.0       1         1
                                     Anderson, Mr. Harry 48.0       1         1
                           Eustis, Miss. Elizabeth Mussey 54.0       1         1
                  Rothschild, Mrs. Martin (Elizabeth L. Barrett) 54.0       1         1
                 Hippach, Mrs. Louis Albert (Ida Sophia Fischer) 44.0       1         1
Duff Gordon, Lady. (Lucille Christiana Sutherland) ("Mrs Morgan") 48.0       1         1
                    Appleton, Mrs. Edward Dale (Charlotte Lamson) 53.0       1         1
                          Frolicher-Stehli, Mr. Maxmillian 60.0       1         1
                 Stephenson, Mrs. Walter Bertram (Martha Eustis) 52.0       1         1
                    Duff Gordon, Sir. Cosmo Edmund ("Mr Morgan") 49.0       1         1
                             Kimball, Mr. Edwin Nelson Jr 42.0       1         1
                      Barkworth, Mr. Algernon Henry Wilson 80.0       1         1
                                Harper, Mr. Henry Sleeper 48.0       1         1
```

```python
# Narmadha k
# 241801181
# 11. Which male passengers paid a fare greater than 100?
# ------------------------------------------------------------

male_fare100 = df[
    (df["Sex"] == "male") &
    (df["Fare"] > 100)
]

print("\n11. Male Passengers Who Paid Fare > 100:")
print(male_fare100[
    ["Name", "Fare", "Sex"]
].to_string(index=False))
```

```
11. Male Passengers Who Paid Fare > 100:
                                    Name     Fare  Sex
             Fortune, Mr. Charles Alexander 263.0000 male
                  Baxter, Mr. Quigg Edmond 247.5208 male
             Allison, Master. Hudson Trevor 151.5500 male
                  Graham, Mr. George Edward 153.4625 male
                        Ringhini, Mr. Sante 135.6333 male
                 Widener, Mr. Harry Elkins 211.5000 male
                Carter, Mr. William Ernest 120.0000 male
                          Fortune, Mr. Mark 263.0000 male
Penasco y Castellana, Mr. Victor de Satode 108.9000 male
                       Farthing, Mr. John 221.7792 male
               Douglas, Mr. Walter Donald 106.4250 male
              Thayer, Mr. John Borland Jr 110.8833 male
                        Robbins, Mr. Victor 227.5250 male
               Newell, Mr. Arthur Webster 113.2750 male
            Frauenthal, Dr. Henry William 133.6500 male
      Cardeza, Mr. Thomas Drake Martinez 512.3292 male
                 Thayer, Mr. John Borland 110.8833 male
                   Lesurer, Mr. Gustave J 512.3292 male
     Carter, Master. William Thornton II 120.0000 male
```

```
# Narmadha k
# 241801181
# 12. List passengers who embarked from port C
#     and were in Class 2
# ------------------------------------------------------------

portC_class2 = df[
    (df["Embarked"] == "C") &
    (df["Pclass"] == 2)
]

print("\n12. Passengers Embarked from C and in Class 2:")
print(portC_class2[
    ["Name", "Embarked", "Pclass"]
].to_string(index=False))
```

```
12. Passengers Embarked from C and in Class 2:
                                                Name Embarked  Pclass
                    Nasser, Mrs. Nicholas (Adele Achem)        C       2
               Laroche, Miss. Simonne Marie Anne Andree        C       2
                                Nasser, Mr. Nicholas        C       2
                                  Richard, Mr. Emile        C       2
                                    Pernot, Mr. Rene        C       2
                                Levy, Mr. Rene Jacques        C       2
                                   Abelson, Mr. Samuel        C       2
                               del Carlo, Mr. Sebastiano        C       2
                                   Lehmann, Miss. Bertha        C       2
          Jerwan, Mrs. Amin S (Marie Marthe Thuillard)        C       2
                               Padro y Manent, Mr. Julian        C       2
Laroche, Mrs. Joseph (Juliette Marie Louise Lafargue)        C       2
                  Laroche, Mr. Joseph Philippe Lemercier        C       2
                                    Mallet, Mr. Albert        C       2
                                  Mallet, Master. Andre        C       2
                          Duran y More, Miss. Asuncion        C       2
                   Abelson, Mrs. Samuel (Hannah Wizosky)        C       2
```

```
# Narmadha k
# 241801181
# 13. Passengers who had more than 2 siblings/spouses
#     aboard (SibSp > 2)
# ------------------------------------------------------------

sibsp_more2 = df[df["SibSp"] > 2]

print("\n13. Passengers with SibSp > 2:")
print(sibsp_more2[
    ["Name", "SibSp"]
].to_string(index=False))
```

```
13. Passengers with SibSp > 2:
                                          Name  SibSp
                        Palsson, Master. Gosta Leonard      3
                                 Rice, Master. Eugene      4
                         Palsson, Miss. Torborg Danira      3
                        Fortune, Mr. Charles Alexander      3
                            Panula, Master. Juha Niilo      4
                    Goodwin, Master. William Frederick      5
                               Skoog, Master. Harald      3
                      Andersson, Miss. Erna Alexandra      4
                           Goodwin, Miss. Lillian Amy      5
Backstrom, Mrs. Karl Alfred (Maria Mathilda Gustafsson)      3
                           Fortune, Miss. Mabel Helen      3
                    Andersson, Miss. Ellis Anna Maria      4
                           Sage, Master. Thomas Henry      8
                        Panula, Master. Eino Viljami      4
                                 Rice, Master. Arthur      4
                       Lefebre, Master. Henry Forbes      3
                        Sage, Miss. Constance Gladys      8
                Asplund, Master. Clarence Gustaf Hugo      4
                                  Sage, Mr. Frederick      8
                              Lefebre, Miss. Mathilde      3
                       Asplund, Miss. Lillian Gertrud      4
                    Asplund, Master. Edvin Rojj Felix      4
                           Panula, Mr. Ernesti Arvid      4
                                 Rice, Master. Eric      4
                            Sage, Mr. George John Jr      8
                       Fortune, Miss. Alice Elizabeth      3
                           Palsson, Miss. Stina Viola      3
```

```
[19]
✓ 0s
```

```python
# Narmadha k
# 241801181
# 14. Passengers who did not survive and had no family
#     aboard (SibSp = 0 and Parch = 0)
# ------------------------------------------------------------

no_survive_no_family = df[
    (df["Survived"] == 0) &
    (df["SibSp"] == 0) &
    (df["Parch"] == 0)
]

print("\n14. Passengers Who Did Not Survive and Had No Family:")
print(no_survive_no_family[
    ["Name", "Survived", "SibSp", "Parch"]
].to_string(index=False))
```

```
                              Mack, Mrs. (Mary)         0      0      0
                                Elias, Mr. Dibo         0      0      0
      Myhrman, Mr. Pehr Fabian Oliver Malkolm         0      0      0
                               Tobin, Mr. Roger         0      0      0
                     Kilgannon, Mr. Thomas J            0      0      0
                      Long, Mr. Milton Clyde          0      0      0
                             Ali, Mr. William         0      0      0
           Harmer, Mr. Abraham (David Lishin)         0      0      0
                     Guggenheim, Mr. Benjamin         0      0      0
                     Keane, Mr. Andrew "Andy"         0      0      0
                          Gaskell, Mr. Alfred         0      0      0
                     Hoyt, Mr. William Fisher         0      0      0
                        Dantcheff, Mr. Ristiu         0      0      0
                           Otter, Mr. Richard         0      0      0
                Ibrahim Shawah, Mr. Yousseff         0      0      0
                       Ponesell, Mr. Martin          0      0      0
                   Johansson, Mr. Karl Johan         0      0      0
                      Andrews, Mr. Thomas Jr         0      0      0
              Pettersson, Miss. Ellen Natalia        0      0      0
                           Meyer, Mr. August         0      0      0
                     Alexander, Mr. William          0      0      0
                           Lester, Mr. James         0      0      0
                Slemen, Mr. Richard James            0      0      0
               Tomlin, Mr. Ernest Portage            0      0      0
                         Fry, Mr. Richard            0      0      0
           Heininen, Miss. Wendla Maria              0      0      0
      Holm, Mr. John Fredrik Alexander               0      0      0
       Reuchlin, Jonkheer. John George               0      0      0
```

riables Terminal

```
                          Montvila, Rev. Juozas      0      0      0
                            Dooley, Mr. Patrick      0      0      0
```

```
20]
✓ 0s
```

```python
# Narmadha k
# 241801181
# 15. Find the top 5 oldest passengers who survived
# ------------------------------------------------------------

oldest_survived = df[
    df["Survived"] == 1
].sort_values(
    by="Age",
    ascending=False
).head(5)

print("\n15. Top 5 Oldest Passengers Who Survived:")
print(oldest_survived[
    ["Name", "Age", "Survived"]
].to_string(index=False))
```

```
15. Top 5 Oldest Passengers Who Survived:
                                    Name  Age  Survived
     Barkworth, Mr. Algernon Henry Wilson 80.0         1
        Andrews, Miss. Kornelia Theodosia 63.0         1
                    Turkula, Mrs. (Hedwig) 63.0        1
                        Harris, Mr. George 62.0        1
Stone, Mrs. George Nelson (Martha Evelyn) 62.0         1
```

```
Stone, Mrs. George Nelson (Martha Evelyn)  62.0          1
```

```
# Narmadha k
# 241801181
# 16. List passengers who paid zero fare
# ------------------------------------------------------------

zero_fare = df[df["Fare"] == 0]

print("\n16. Passengers Who Paid Zero Fare:")
print(zero_fare[
    ["Name", "Fare"]
].to_string(index=False))
```

```
16. Passengers Who Paid Zero Fare:
                            Name  Fare
            Leonard, Mr. Lionel   0.0
          Harrison, Mr. William   0.0
    Tornquist, Mr. William Henry   0.0
     Parkes, Mr. Francis "Frank"   0.0
 Johnson, Mr. William Cahoone Jr   0.0
  Cunningham, Mr. Alfred Fleming   0.0
          Campbell, Mr. William   0.0
Frost, Mr. Anthony Wood "Archie"   0.0
            Johnson, Mr. Alfred   0.0
   Parr, Mr. William Henry Marsh   0.0
      Watson, Mr. Ennis Hastings   0.0
           Knight, Mr. Robert J   0.0
         Andrews, Mr. Thomas Jr   0.0
               Fry, Mr. Richard   0.0
 Reuchlin, Jonkheer. John George   0.0
```

```
# Narmadha k
# 241801181
# 17. Split the dataset for training and testing
# ------------------------------------------------------------

# Features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]

# Select numerical columns for a simple train-test split
X_numeric = df[
    ["Pclass", "Age", "SibSp", "Parch", "Fare"]
]

# Split into 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(
    X_numeric,
    y,
    test_size=0.2,
    random_state=42
)

print("\n17. Train-Test Split:")
print("Training data:", X_train.shape)
print("Testing data :", X_test.shape)
print("Training target:", y_train.shape)
print("Testing target :", y_test.shape)
```

```
17. Train-Test Split:
Training data: (712, 5)
Testing data : (179, 5)
Training target: (712,)
Testing target : (179,)
```

```
# 241801181
# FINAL DATASET
# ------------------------------------------------------------

print("\nFinal Dataset:")
print(df.head())

print("\nMissing Values After Preprocessing:")
print(df.isnull().sum())
```

```
Final Dataset:
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare    Cabin Embarked
0      0         A/5 21171   7.2500  Unknown        S
1      0          PC 17599  71.2833      C85        C
2      0  STON/O2. 3101282   7.9250  Unknown        S
3      0            113803  53.1000     C123        S
4      0            373450   8.0500  Unknown        S

Missing Values After Preprocessing:
PassengerId    0
Survived       0
Pclass         0
Name           0
Sex            0
Age            0
SibSp          0
Parch          0
Ticket         0
Fare           0
Cabin          0
```

ds exp:3

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[1] ✓ 1s

```
# Narmadha K
# 241801181
# Step 1: Import required libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVR
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

[4] ✓ 1m

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

[5] ✓ 10s

```
# Narmadha K
# 241801181
# Step 2: Upload and load the Advertising dataset
from google.colab import files
uploaded = files.upload()
df = pd.read_csv("Advertising.csv")
print("First 5 rows:")
display(df.head())
```

Choose Files Advertising.csv

Advertising.csv(text/csv) - 5166 bytes, last modified: 9/11/2026 - 100% done

```
Saving Advertising.csv to Advertising.csv
First 5 rows:
```

	Unnamed: 0	TV	Radio	Newspaper	Sales
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9

```
# Narmadha K
# 241801181
# Step 3: Explore and preprocess the data
print("Dataset Information:")
df.info()
print("\nNull Values:")
print(df.isnull().sum())
print("\nData Types:")
print(df.dtypes)
print("\nStatistical Summary:")
display(df.describe())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Unnamed: 0   200 non-null    int64
 1   TV           200 non-null    float64
 2   Radio        200 non-null    float64
 3   Newspaper    200 non-null    float64
 4   Sales        200 non-null    float64
dtypes: float64(4), int64(1)
memory usage: 7.9 KB

Null Values:
Unnamed: 0    0
TV            0
Radio         0
Newspaper     0
Sales         0
dtype: int64

Data Types:
Unnamed: 0      int64
TV            float64
Radio         float64
Newspaper     float64
Sales         float64
dtype: object

Statistical Summary:
```

	Unnamed: 0	TV	Radio	Newspaper	Sales
count	200.000000	200.000000	200.000000	200.000000	200.000000
mean	100.500000	147.042500	23.264000	30.554000	14.022500
std	57.879185	85.854236	14.846809	21.778621	5.217457

min	1.000000	0.700000	0.000000	0.300000	1.600000
25%	50.750000	74.375000	9.975000	12.750000	10.375000
50%	100.500000	149.750000	22.900000	25.750000	12.900000
75%	150.250000	218.825000	36.525000	45.100000	17.400000
max	200.000000	296.400000	49.600000	114.000000	27.000000

[7]
0s

```
# Narmadha K
# 241801181
# Step 4: Select features and target
X = df[["TV", "Radio", "Newspaper"]]
y = df["Sales"]
print("Features:")
display(X.head())
print("\nTarget:")
display(y.head())
```

```
Features:
```

	TV	Radio	Newspaper
0	230.1	37.8	69.2
1	44.5	39.3	45.1
2	17.2	45.9	69.3
3	151.5	41.3	58.5
4	180.8	10.8	58.4

```
Target:
```

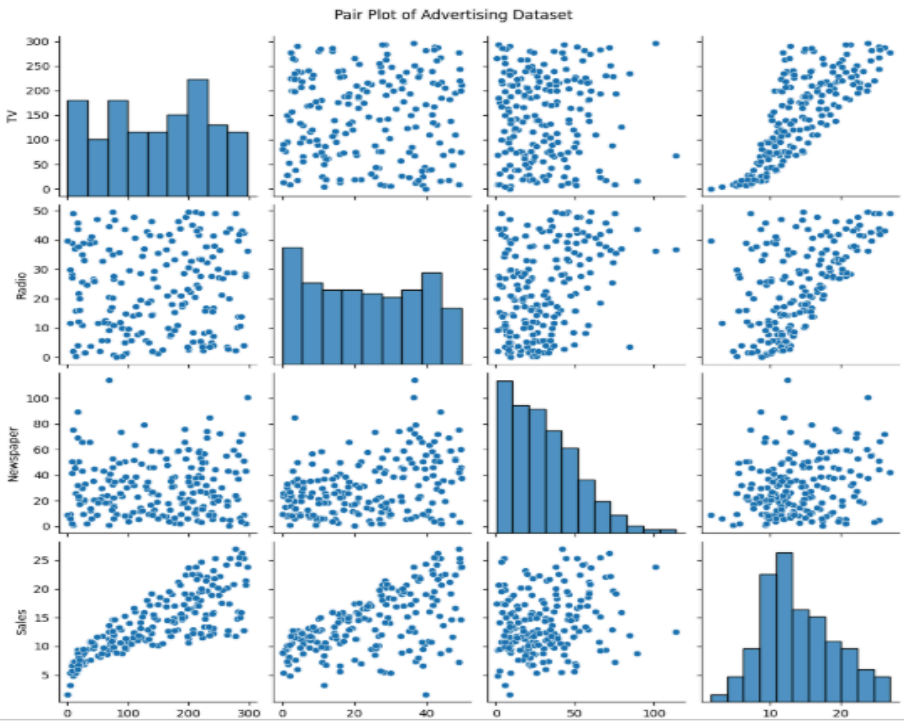
	Sales
0	22.1
1	10.4
2	9.3
3	18.5
4	12.9

```
dtype: float64
```

Commands + Code + Text | Run all

[8]
2s

```
# Narmadha K
# 241801181
# Step 5: Create pair plot for features and target
sns.pairplot(df[["TV", "Radio", "Newspaper", "Sales"]])
plt.suptitle("Pair Plot of Advertising Dataset", y=1.02)
plt.show()
```



[9]
0s

```
# Narmadha K
# 241801181
# Step 6: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Training data shape: (160, 3)
Testing data shape: (40, 3)
```

TV Radio Newspaper

[9] 0s

```
# Narmadha K
# 241801181
# Step 6: Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Training data shape: (160, 3)
Testing data shape: (40, 3)
```

[10] 0s

```
# Narmadha K
# 241801181
# Step 7: Train Linear Regression model
linear_model = LinearRegression()
# Fit the model
linear_model.fit(X_train, y_train)
# Predict on test data
y_pred_linear = linear_model.predict(X_test)
print("Linear Regression Predictions:")
print(y_pred_linear[:10])
```

```
Linear Regression Predictions:
[16.4080242  20.88988209 21.55384318 10.60850256 22.11237326 13.10559172
 21.05719192  7.46101034 13.60634581 15.15506967]
```

[11] 0s

```
# Narmadha K
# 241801181
# Step 8: Train Polynomial Regression with degree = 2
polynomial_model = Pipeline([
    ("poly", PolynomialFeatures(degree=2)),
    ("linear", LinearRegression())
])
# Fit the model
polynomial_model.fit(X_train, y_train)
# Predict on test data
y_pred_poly = polynomial_model.predict(X_test)
print("Polynomial Regression Predictions:")
print(y_pred_poly[:10])
```

```
Polynomial Regression Predictions:
[17.25443578 22.7193321  20.42799436  7.542709   24.37103037 12.55927164
 22.80784986  8.34371358 12.0586023  15.69024565]
```

[12] 0s

```
# Narmadha K
# 241801181
# Step 9: Train Support Vector Regression with RBF kernel
svr_model = Pipeline([
    ("scaler", StandardScaler()),
    ("svr", SVR(kernel="rbf"))
])
# Fit the model
svr_model.fit(X_train, y_train)
# Predict on test data
y_pred_svr = svr_model.predict(X_test)
print("SVR Predictions:")
print(y_pred_svr[:10])
```

```
SVR Predictions:
[17.1879597  21.60497277 19.5802057   9.76637843 19.9549199  12.49691541
 21.90662014  8.54092114 12.05397226 15.48663882]
```

[13] 0s

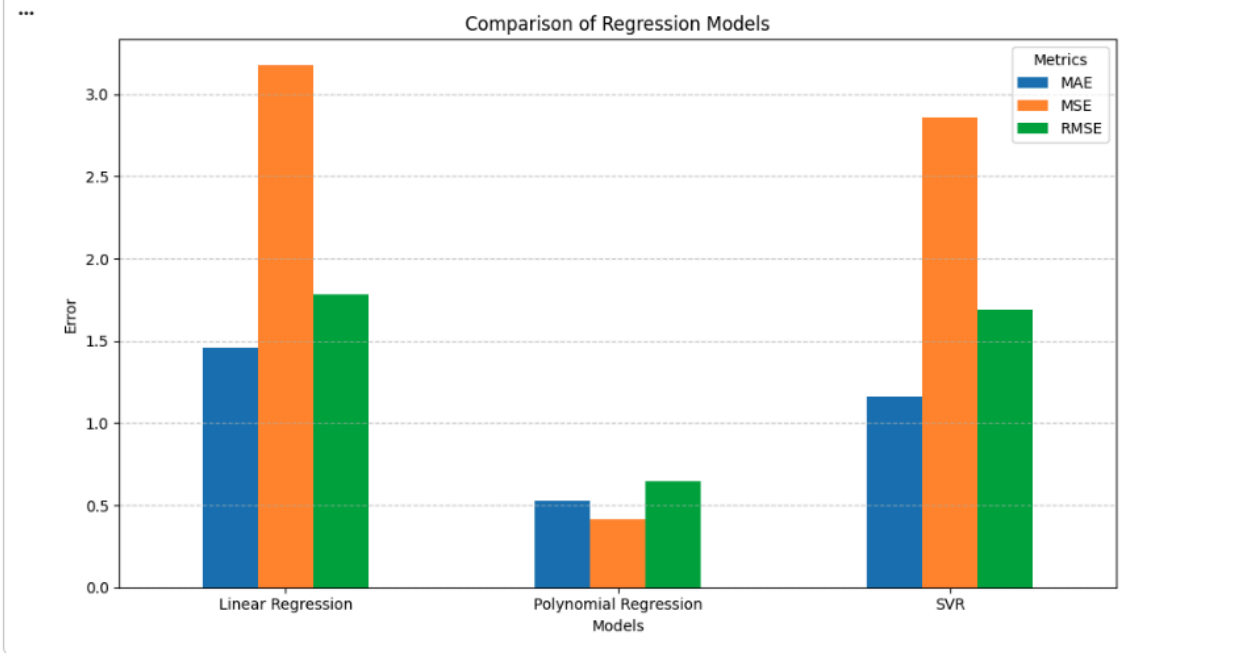
```
# Narmadha K
# 241801181
# Step 10: Calculate MAE, MSE and RMSE for each model
def evaluate_model(y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    return mae, mse, rmse
# Linear Regression
mae_linear, mse_linear, rmse_linear = evaluate_model(
    y_test, y_pred_linear
)
# Polynomial Regression
mae_poly, mse_poly, rmse_poly = evaluate_model(
    y_test, y_pred_poly
)
# SVR
mae_svr, mse_svr, rmse_svr = evaluate_model(
    y_test, y_pred_svr
)
results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Polynomial Regression",
        "SVR"
    ],
    "MAE": [
        mae_linear,
        mae_poly,
```

```
    "Model": [
        "Linear Regression",
        "Polynomial Regression",
        "SVR"
    ],
    "MAE": [
        mae_linear,
        mae_poly,
        mae_svr
    ],
    "MSE": [
        mse_linear,
        mse_poly,
        mse_svr
    ],
    "RMSE": [
        rmse_linear,
        rmse_poly,
        rmse_svr
    ]
})
display(results)
```

	Model	MAE	MSE	RMSE
0	Linear Regression	1.460757	3.174097	1.781600
1	Polynomial Regression	0.526179	0.412910	0.642581
2	SVR	1.162139	2.862109	1.691777

Next steps: Generate code with results | New interactive sheet

```
# Narmadha K
# 241801181
# Step 11: Compare all models using a single bar graph
results.set_index("Model")[["MAE", "MSE", "RMSE"]].plot(
    kind="bar",
    figsize=(10, 6)
)
plt.title("Comparison of Regression Models")
plt.xlabel("Models")
plt.ylabel("Error")
plt.xticks(rotation=0)
plt.legend(title="Metrics")
plt.grid(axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
plt.show()
```



File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

[15] ✓ 0s

```
# Narmadha K
# 241801181
# Step 12: Apply K-Means clustering with 3 clusters
features = df[["TV", "Radio", "Newspaper"]]
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
# Fit K-Means and assign cluster labels
df["Cluster"] = kmeans.fit_predict(features)
print("Dataset with Cluster Labels:")
display(df.head())
print("\nCluster Counts:")
print(df["Cluster"].value_counts())
```

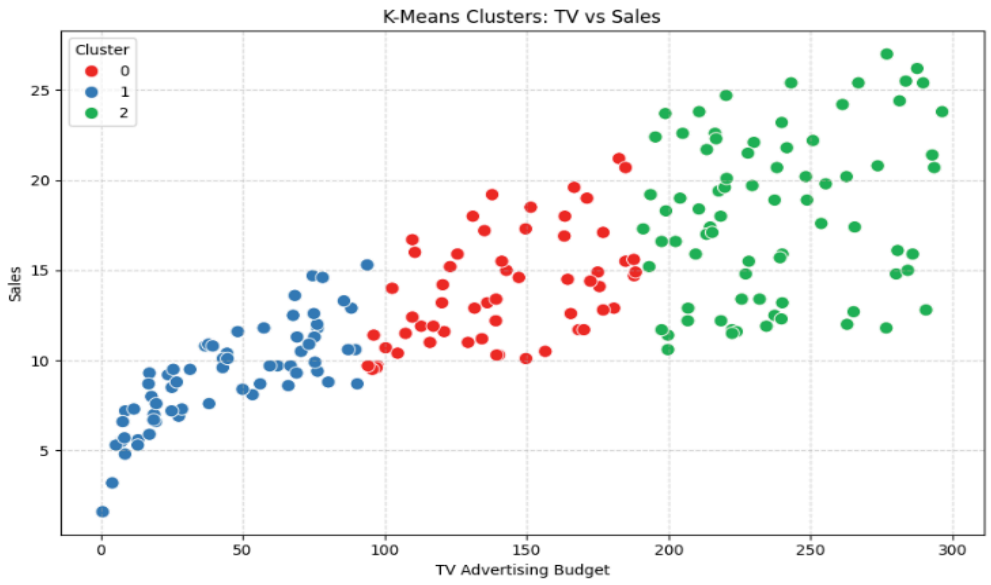
```
Dataset with Cluster Labels:
```

	Unnamed: 0	TV	Radio	Newspaper	Sales	Cluster
0	1	230.1	37.8	69.2	22.1	2
1	2	44.5	39.3	45.1	10.4	1
2	3	17.2	45.9	69.3	9.3	1
3	4	151.5	41.3	58.5	18.5	0
4	5	180.8	10.8	58.4	12.9	0

```
Cluster Counts:
Cluster
2    77
1    65
0    58
Name: count, dtype: int64
```

[16] ✓ 1s

```
# Narmadha K
# 241801181
# Step 13: Visualize K-Means clusters
plt.figure(figsize=(9, 6))
sns.scatterplot(
    data=df,
    x="TV",
    y="Sales",
    hue="Cluster",
    palette="Set1",
    s=80
)
plt.title("K-Means Clusters: TV vs Sales")
plt.xlabel("TV Advertising Budget")
plt.ylabel("Sales")
plt.legend(title="Cluster")
plt.grid(True, linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```



[17]
✓ 0s

```
# Narmadha K
# 241801181
# Step 14: Display final dataset with cluster labels
print("Final Dataset:")
display(df)
```

```
Final Dataset:
```

	Unnamed: 0	TV	Radio	Newspaper	Sales	Cluster
0	1	230.1	37.8	69.2	22.1	2
1	2	44.5	39.3	45.1	10.4	1
2	3	17.2	45.9	69.3	9.3	1
3	4	151.5	41.3	58.5	18.5	0
4	5	180.8	10.8	58.4	12.9	0
...	...	...	...	...	...	...
195	196	38.2	3.7	13.8	7.6	1
196	197	94.2	4.9	8.1	9.7	0
197	198	177.0	9.3	6.4	12.8	0
198	199	283.6	42.0	66.2	25.5	2
199	200	232.1	8.6	8.7	13.4	2

200 rows × 6 columns

Next steps: Generate code with df | New interactive sheet

[18]
✓ 0s

```
# Narmadha K
# 241801181
print(uploaded.keys())
```

```
dict_keys(['Advertising.csv'])
```

[45]
```python
# Name: Narmadha K
# Roll No: 241801181
# Step 1: Import Libraries

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVR
from sklearn.cluster import KMeans
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

[46]
```python
# Name: Narmadha K
# Roll No: 241801181
# Step 2: Load Dataset
df = pd.read_csv("house_prices.csv")
print(df.head())
```

```
   Id  Area  Bedrooms  Bathrooms  Floors  YearBuilt  Location  Condition  \
0   1  1360         5          4       3       1970  Downtown  Excellent
1   2  4272         5          4       3       1958  Downtown  Excellent
2   3  3592         2          2       3       1938  Downtown       Good
3   4   966         4          2       2       1902  Suburban       Fair
4   5  4926         1          4       2       1975  Downtown       Fair

  Garage   Price
0     No  149919
1     No  424998
2     No  266746
3    Yes  244020
4    Yes  636056
```

Commands + Code + Text ▷ Run all

[47]
```python
# Name: Narmadha K
# Roll No: 241801181
# Step 3: Explore Dataset
print("Dataset Information:")
df.info()
print("\nStatistical Summary:")
print(df.describe())
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 10 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Id         2000 non-null   int64
 1   Area       2000 non-null   int64
 2   Bedrooms   2000 non-null   int64
 3   Bathrooms  2000 non-null   int64
 4   Floors     2000 non-null   int64
 5   YearBuilt  2000 non-null   int64
 6   Location   2000 non-null   object
 7   Condition  2000 non-null   object
 8   Garage     2000 non-null   object
 9   Price      2000 non-null   int64
dtypes: int64(7), object(3)
memory usage: 156.4+ KB

Statistical Summary:
                Id         Area     Bedrooms   Bathrooms       Floors  \
count  2000.000000  2000.000000  2000.000000  2000.00000  2000.000000
mean   1000.500000  2786.209500     3.003500     2.55250     1.993500
std     577.494589  1295.146799     1.424606     1.10899     0.809188
min       1.000000   501.000000     1.000000     1.00000     1.000000
25%     500.750000  1653.000000     2.000000     2.00000     1.000000
50%    1000.500000  2833.000000     3.000000     3.00000     2.000000
75%    1500.250000  3887.500000     4.000000     4.00000     3.000000
max    2000.000000  4999.000000     5.000000     4.00000     3.000000

         YearBuilt          Price
count  2000.000000    2000.000000
mean   1961.446000  537676.855000
std      35.926695  276428.845719
min    1900.000000   50005.000000
25%    1930.000000  300098.000000
50%    1961.000000  539254.000000
75%    1993.000000  780086.000000
max    2023.000000  999656.000000
```

[48]
✓ 0s

```
# Name: Narmadha K
# Roll No: 241801181
# Step 4: Check Missing Values
print(df.isnull().sum())
```

```
Id            0
Area          0
Bedrooms      0
Bathrooms     0
Floors        0
YearBuilt     0
Location      0
Condition     0
Garage        0
Price         0
dtype: int64
```

[49]
✓ 0s

```
# Name: Narmadha K
# Roll No: 241801181
# Step 5: Select Features and Target
-X = df[['Area', 'Bedrooms', 'Age', 'Distance']]
+
+# Calculate 'Age' from 'YearBuilt'
+current_year = 2023
+df['Age'] = current_year - df['YearBuilt']
+
+X = df[['Area', 'Bedrooms', 'Age']]
y = df['Price']
print("Features:")
print(X.head())
print("\nTarget:")
print(y.head())
```

```
Features:
   Area  Bedrooms  Age
0  1360         5   53
1  4272         5   65
2  3592         2   85
3   966         4  121
4  4926         1   48

Target:
0    149919
1    424998
2    266746
3    244020
4    636056
Name: Price, dtype: int64
```

[50]
✓ 0s

```
# Name: Narmadha K
# Roll No: 241801181
# Step 6: Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("Training Data:", X_train.shape)
print("Testing Data:", X_test.shape)
```

```
Training Data: (1600, 3)
Testing Data: (400, 3)
```

[51]
✓ 0s

```
# Name: Narmadha K
# Roll No: 241801181
# Step 7: Linear Regression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)
print("Linear Regression Predictions:")
print(y_pred_linear)
```

```
 532791.76070132 536505.14501254 540834.16116498 531331.18983308
 543126.56801786 531512.88086508 542346.84673794 536864.99302727
 541490.65943056 541218.60343836 537374.46486667 537569.77946271
 534157.27316644 536345.85301212 529610.9300111  542052.47124873
 532591.90620745 530380.61337663 535608.08741024 533359.25563621
 537719.97201937 539317.59996913 537010.52961737 534084.61214143
 539750.28812331 541113.85145652 533104.30036372 540828.90309147
 529605.06478708 531171.89096924 532664.85467452 531926.18761666
 529429.50062095 530470.7608039  539850.78354798 529965.79437886
 542177.57576743 533064.96583365 532371.98661878 529900.43997277
 533863.90423712 537760.00507701 529657.86933612 533901.76786789
 531234.8102052  532723.76139775 540174.87060895 535676.52841225
 530163.75276384 541452.86089619 536468.99057064 534760.21920293
 532186.81732259 539263.67484218 530452.95768606 535260.55136021
 530281.33999409 542674.83206276 539502.44158491 540597.80695141
 539491.97751878 541874.1403615  539330.15392759 536123.72059898
 537554.58857873 534392.53718416 540934.09677177 530735.81820948
 541190.69132401 539604.63341377 542762.22854647 540255.19560406
 530353.29949867 531760.88094855 533384.11950867 536901.01562289
 542199.10008538 532834.34499782 532467.72666339 535260.57582103
 534948.87157604 540314.73299645 539949.96050988 532663.8175017
 542613.78928759 532312.98529477 531761.91401996 538298.34643817
 532344.84388329 540784.81136149 537419.25717493 541712.86861624
 537849.14893725 529752.13737289 542327.67395416 533088.19546993
 538533.17838289 531807.84473439 541619.5018046  531440.60259421
 529598.70373305 533866.16732262 530317.76886251 541250.81527665
```

[52]

```
# Name: Narmadha K
# Roll No: 241801181
# Step 8: Polynomial Regression
poly_model = Pipeline([
    ('polynomial', PolynomialFeatures(degree=2)),
    ('linear', LinearRegression())
])
poly_model.fit(X_train, y_train)
y_pred_poly = poly_model.predict(X_test)
print("Polynomial Regression Predictions:")
print(y_pred_poly)
```

```
 537963.65057467 519969.3242574  551617.13532972 519951.69075035
 552042.91977643 536445.12091883 545521.19286351 556701.23365804
 555038.58357842 537319.1357773  530226.93144501 541070.80434564
 554371.00409551 548746.24413497 553528.5803476  534957.93903965
 533518.65047637 508945.34502109 545506.8451735  527379.11706778
 548243.86820344 543342.16474369 534073.73326972 536242.41080276
 557888.84432707 559438.68787269 557359.17436625 543073.66410041
 534347.65279667 562268.87222277 558842.09225675 552104.15223623
 560766.85854235 531428.49760451 536092.51556964 520424.16105689
 526020.44933287 544665.84049458 539047.16670751 553717.57901568
 544689.95128603 551273.85099918 521171.78481876 535736.55260493
 512703.07699085 550866.57744604 556157.57141518 542385.98615533
 547578.23139557 542344.55163633 544608.32305537 563484.32861884
 529021.69516554 531933.87195309 504446.15895106 558555.13794388
 553702.07413123 543562.72516027 556457.21323785 513174.92338961
 523302.59965305 560181.34658724 520616.51135515 513777.21888599
 521874.97366606 543745.4690136  543080.49442392 528229.18608553
 538250.9051938  529201.65226564 563307.52251502 537968.73445527
 514960.1381799  511123.24194034 533277.52943703 538463.83493584
 519567.3590101B 539113.64521756 517190.16377133 554330.85592074
 549184.38792196 519352.38891138 542708.81680551 539171.11348677
 496630.53460852 558910.83132956 516480.78626134 552614.42515774
 520382.75758437 528818.46545382 548256.08446291 546144.48885299
 573004.66741145 543364.96660511 534026.97654322 515798.30115748
 562008.66093758 532223.61991404 520798.12418481 560713.35243384
 536277.8377125  542474.50851749 523313.83125489 534194.52418863
 541303.91261485 527254.88636799 522512.12722771 544716.54571981
 548419.98499081 547536.97703346 525384.63824964 513473.51856644
 543687.14445931 567131.81231403 529460.39139755 554156.10008643
 542831.919781   563215.34789135 485458.95151092 523326.23995501
 525726.65734109 559840.26810419 550417.84764251 513554.40031965
 543453.86870733 540918.76574506 542416.9396263  537134.55343845
 557412.63583864 510089.38734288 559762.64057606 540900.68518154
 527791.42652844 567190.00813212 535502.03581842 539001.07683396
 537297.75625201 485413.10731662 514055.25723873 516451.08249542
 538663.95824213 508188.40888941 544237.13765134 543325.05615898
 537814.89856748 554850.30709608 551675.61792356 531451.61181024
 551701.75222761 498378.36907899 524987.48437348 543152.36532513
 528355.12308471 536117.99708505 555506.13451575 543054.54042758
```

[53]

```
# Name: Narmadha K
# Roll No: 241801181
# Step 9: Support Vector Regression
svr_model = Pipeline([
    ('scaler', StandardScaler()),
    ('svr', SVR(kernel='rbf'))
])
svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)
print("SVR Predictions:")
print(y_pred_svr)
```

```
 533469.06357139 533465.90959776 533463.57915614 533464.37709917
 533460.07345275 533449.95172212 533461.81836961 533471.62393548
 533459.49768992 533460.35888645 533468.9045686  533470.51331784
 533456.08990379 533455.64883208 533460.6814886  533462.21214135
 533460.9808676  533466.52587711 533472.0439912  533464.55493451
 533464.63343686 533452.23359287 533455.56979934 533463.34401155
 533462.8198705  533454.98062726 533471.19545908 533459.7127856
 533463.21673131 533466.71132989 533469.46582013 533461.95988908
 533460.46998957 533460.26320664 533462.71000285 533465.94173441
 533459.98955899 533463.91856462 533473.5253693  533456.94111196
 533450.29847537 533466.16019236 533463.36738066 533470.47384895
 533469.11165339 533464.16412922 533463.27995664 533464.82185558
 533467.4859101  533463.86195972 533466.10840286 533473.19535797
 533472.4844786  533453.63795419 533469.90627881 533471.86409113
 533465.78413739 533457.44269024 533459.66320821 533465.6835594
 533454.64217994 533460.36670255 533455.14473159 533471.43228372
 533470.77908875 533468.99553186 533456.94512257 533471.59997349
 533460.64747491 533468.91952917 533463.02158322 533461.84956696
 533454.22236736 533459.5559909  533463.9011008  533471.14202881
 533455.46243009 533462.66105961 533463.95440278 533454.75599564
 533466.29241453 533462.12249234 533462.60609087 533449.57856165
 533461.94772223 533459.48218763 533465.44994977 533464.21857327
 533453.4173714  533466.17057117 533469.35482782 533456.1316246
 533457.4830765  533457.99123119 533459.92071238 533469.13071148
 533452.22019808 533462.69947582 533459.13510283 533464.04526125
 533460.29431305 533462.05741164 533458.22300534 533464.27264447
 533451.22765656 533463.6722044  533466.3628005  533460.96700181
 533464.87739547 533462.59753062 533470.35807162 533462.25241215
 533459.88911357 533459.57061942 533451.38915533 533460.68487882
 533458.38269977 533453.67040007 533460.91233503 533472.78476969
 533454.50487651 533471.00426745 533462.03356166 533464.28570233
 533458.11008719 533459.0870155  533472.78446041 533470.82849265
 533456.3328836  533463.53279634 533461.62207469 533472.4351391
 533464.57263683 533469.00910945 533453.63486732 533468.67154762
 533461.60998798 533466.87925584 533465.44714964 533464.57648857
```

Show command palette (Ctrl+Shift+P)

```
# Name: Narmadha K
# Roll No: 241801181
# Step 10: Evaluate Models
models = {
    "Linear Regression": y_pred_linear,
    "Polynomial Regression": y_pred_poly,
    "SVR": y_pred_svr
}
results = []
for name, prediction in models.items():
    mae = mean_absolute_error(y_test, prediction)
    mse = mean_squared_error(y_test, prediction)
    rmse = np.sqrt(mse)
    print("\nModel:", name)
    print("MAE :", mae)
    print("MSE :", mse)
    print("RMSE:", rmse)
    results.append([name, mae, mse, rmse])
```

```
Model: Linear Regression
MAE : 242644.90318882384
MSE : 77946329305.06284
RMSE: 279188.69838348194

Model: Polynomial Regression
MAE : 244080.05753596313
MSE : 78191989752.78658
RMSE: 279628.306422627

Model: SVR
MAE : 242647.62272976793
MSE : 77902507680.36156
RMSE: 279110.2070515544
```

[38]

```
# Name: Narmadha K
# Roll No: 241801181
# Step 11: Model Comparison
results_df = pd.DataFrame(
    results,
    columns=["Model", "MAE", "MSE", "RMSE"]
)
print(results_df)
```

```
                        Model            MAE            MSE            RMSE
0      Linear Regression  242644.903189  7.794633e+10  279188.698383
1  Polynomial Regression  244080.057536  7.819199e+10  279628.306423
2                    SVR  242647.622730  7.790251e+10  279110.207052
```

RMSE: 279110.2070515544

[38]

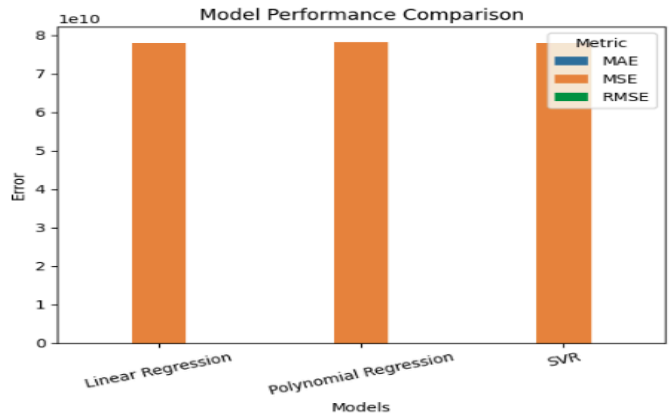
```
# Name: Narmadha K
# Roll No: 241801181
# Step 11: Model Comparison
results_df = pd.DataFrame(
    results,
    columns=["Model", "MAE", "MSE", "RMSE"]
)
print(results_df)
```

```
                        Model            MAE            MSE            RMSE
0      Linear Regression  242644.903189  7.794633e+10  279188.698383
1  Polynomial Regression  244080.057536  7.819199e+10  279628.306423
2                    SVR  242647.622730  7.790251e+10  279110.207052
```

[55]

```
# Name: Narmadha K
# Roll No: 241801181
# Step 12: Compare Models using Bar Graph
results_melted = results_df.melt(
    id_vars="Model",
    value_vars=["MAE", "MSE", "RMSE"],
    var_name="Metric",
    value_name="Error"
)
sns.barplot(
    data=results_melted,
    x="Model",
    y="Error",
    hue="Metric"
)
plt.title("Model Performance Comparison")
plt.xlabel("Models")
plt.ylabel("Error")
plt.xticks(rotation=15)
plt.show()
```

```
plt.show()
```



Gemini

```
# Name: Narmadha K
# Roll No: 241801181
# Step 13: K-Means Clustering
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
kmeans.fit(X)
df['Cluster'] = kmeans.labels_
print("Cluster Labels:")
-print(df[['Area', 'Bedrooms', 'Age', 'Distance', 'Price', 'Cluster']])
+print(df[['Area', 'Bedrooms', 'Age', 'Price', 'Cluster']])
```

```
Cluster Labels:
      Area  Bedrooms  Age   Price  Cluster
0     1360         5   53  149919        0
1     4272         5   65  424998        1
2     3592         2   85  266746        1
3      966         4  121  244020        0
4     4926         1   48  636056        1
```

```
# Name: Narmadha K
# Roll No: 241801181
# Step 14: Visualize Clusters
sns.scatterplot(
    data=df,
    x='Area',
    y='Price',
    hue='Cluster'
)
plt.xlabel("House Area")
plt.ylabel("House Price")
plt.title("House Price Clusters")
plt.show()
```




```
# Name: Narmadha K
# Roll No: 241801181
# Step 15: Find Best Model
best_model = results_df.loc[
    results_df['RMSE'].idxmin()
]
print("Best Model:", best_model['Model'])
print("Lowest RMSE:", best_model['RMSE'])
```

```
Best Model: SVR
Lowest RMSE: 279110.2070515544
```

```
# Narmadha K
# 241801181
# Step 1: Install Required Libraries

!pip install -q kagglehub nltk scikit-learn pandas
```

```
# Narmadha K
# 241801181
# Step 2: Import Required Libraries

import os
import re
import pandas as pd
import nltk
import kagglehub

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
# Narmadha K
# 241801181
# Step 3: Download NLTK Resources
```

```
# 241801181
# Step 3: Download NLTK Resources

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
# Narmadha K
# 241801181
# Step 4: Download Amazon Fine Food Reviews Dataset

path = kagglehub.dataset_download(
    "snap/amazon-fine-food-reviews"
)

print("Dataset downloaded to:", path)
```

```
Using Colab cache for faster access to the 'amazon-fine-food-reviews' dataset.
Dataset downloaded to: /kaggle/input/amazon-fine-food-reviews
```

```python
# Narmadha K
# 241801181
# Step 5: Load the Dataset

csv_file = None

for root, dirs, files in os.walk(path):
    for file in files:
        if file.endswith(".csv"):
            csv_file = os.path.join(root, file)
            break
    if csv_file:
        break

print("CSV file:", csv_file)

df = pd.read_csv(csv_file)

print("Original Dataset Shape:", df.shape)
df.head()
```

```
CSV file: /kaggle/input/amazon-fine-food-reviews/Reviews.csv
Original Dataset Shape: (568454, 10)
```

Original Dataset Shape: (568454, 10)

	Id	ProductId	UserId	ProfileName	HelpfulnessNumerator	HelpfulnessDenominator	Score	Time	Summary	Text
0	1	B001E4KFG0	A3SGXH7AUHU8GW	delmartian	1	1	5	1303862400	Good Quality Dog Food	I have bought several of the Vitality canned d...
1	2	B00813GRG4	A1D87F6ZCVE5NK	dll pa	0	0	1	1346976000	Not as Advertised	Product arrived labeled as Jumbo Salted Peanut...
2	3	B000LQOCH0	ABXLMWJIXXAIN	Natalia Corres "Natalia Corres"	1	1	4	1219017600	"Delight" says it all	This is a confection that has been around a fe...
3	4	B000UA0QIQ	A395BORC6FGVXV	Karl	3	3	2	1307923200	Cough Medicine	If you are looking for the secret ingredient i...
4	5	B006K2ZZ7K	A1UQRSCLF8GW1T	Michael D. Bigham "M. Wassir"	0	0	5	1350777600	Great taffy	Great taffy at a great price. There was a wid...

```python
# Narmadha K
# 241801181
# Step 6: Select Review Text, Remove Nulls and Limit to 10,000
df = df[['Text']].copy()
df = df.dropna()
df = df.head(10000)
print("Dataset Shape:", df.shape)
```

```
Dataset Shape: (10000, 1)
```

```python
# Narmadha K
# 241801181
# Step 7: Load English Stopwords
stop_words = set(stopwords.words('english'))
print("Number of Stopwords:", len(stop_words))
print(list(stop_words)[:20])
```

```
Number of Stopwords: 198
["she'll", "shouldn't", "he'll", "it's", "didn't", 'that', 'him', 'myself', 'not', "doesn't", 'yourself', 'a', "i've", 'with', 'other', 'theirs', 'who', 'her', 'didn', 'ourselves']
```

```python
# Narmadha K
# 241801181
# Step 8: Preprocess Review Text
def preprocess_text(text):
    # Convert to lowercase
    text = text.lower()
    # Remove punctuation and special characters
    text = re.sub(r'[^a-zA-Z\s]', '', text)
    # Tokenize
    tokens = word_tokenize(text)
    # Remove stopwords
    tokens = [
        word for word in tokens
        if word not in stop_words
    ]
    # Join tokens
    return ' '.join(tokens)

print("Preprocessing function created successfully.")
```

```
print("Preprocessing function created successfully.")
```

```
Preprocessing function created successfully.
```

```
# Narmadha K
# 241801181
# Step 9: Apply Preprocessing to All Reviews

df['Cleaned_Text'] = df['Text'].apply(preprocess_text)

df.head()
```

	Text	Cleaned_Text
0	I have bought several of the Vitality canned d...	bought several vitality canned dog food produc...
1	Product arrived labeled as Jumbo Salted Peanut...	product arrived labeled jumbo salted peanutsth...
2	This is a confection that has been around a fe...	confection around centuries light pillowy citr...
3	If you are looking for the secret ingredient i...	looking secret ingredient robitussin believe f...
4	Great taffy at a great price. There was a wid...	great taffy great price wide assortment yummy ...

```
# Narmadha K
# 241801181
# Step 10: Convert Text into TF-IDF Vector
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(
    df['Cleaned_Text']
)
print("TF-IDF Matrix Shape:", tfidf_matrix.shape)
```

```
TF-IDF Matrix Shape: (10000, 23530)
```

```
# Narmadha K
# 241801181
# Step 11: Retrieve Documents using Cosine Similarity

def retrieve_documents(query, top_k=5):
    # Preprocess query
    cleaned_query = preprocess_text(query)
    # Convert query to TF-IDF
    query_vector = vectorizer.transform(
        [cleaned_query]
    )
    # Calculate cosine similarity
    similarity_scores = cosine_similarity(
        query_vector,
        tfidf_matrix
    ).flatten()
    # Get top-k results
    top_indexes = similarity_scores.argsort()[-top_k:][::-1]
    print("\nQuery:", query)
    print("Cleaned Query:", cleaned_query)
    print("\nTop", top_k, "Relevant Reviews:\n")
    for rank, index in enumerate(top_indexes, 1):
        print("Rank:", rank)
        print("Similarity Score:",
              round(similarity_scores[index], 4))
        print("\nOriginal Review:")
        print(df.iloc[index]['Text'])
        print("\nCleaned Review:")
        print(df.iloc[index]['Cleaned_Text'])
        print("-" * 80)
```

```python
# Narmadha K
# 241801181
# Step 12: Query - Great Product with Fast Shipping

retrieve_documents(
    "great product with fast shipping",
    top_k=5
)
```

```
Query: great product with fast shipping
Cleaned Query: great product fast shipping

Top 5 Relevant Reviews:

Rank: 1
Similarity Score: 0.5024

Original Review:
Enjoyed the product and they also provided very fast shipping.  I'm about out and need to order more.

Cleaned Review:
enjoyed product also provided fast shipping im need order
--------------------------------------------------------------------------------
Rank: 2
Similarity Score: 0.4439

Original Review:
The tea is good and fresh. We enjoy it. The shipping is fast and cost is reasonable.<br />I recommend this product.

Cleaned Review:
tea good fresh enjoy shipping fast cost reasonablebr recommend product
```

```python
# Narmadha K
# 241801181
# Step 13: Query - Disappointed

retrieve_documents(
    "disappointed",
    top_k=5
)
```

```
Query: disappointed
Cleaned Query: disappointed

Top 5 Relevant Reviews:

Rank: 1
Similarity Score: 0.485

Original Review:
I am a bit disappointed.  The flavor was not what I wanted or expected.

Cleaned Review:
bit disappointed flavor wanted expected
--------------------------------------------------------------------------------
Rank: 2
Similarity Score: 0.3568

Original Review:
The product is very good. Way too expensive and at almost 5.00 a box to get pancakes or biscuits I was disappointed

Cleaned Review:
product good way expensive almost box get pancakes biscuits disappointed
--------------------------------------------------------------------------------
```

Ex5a: INFORMATION EXTRACTION USING NLTK

1. Download Amazon Fine Food Reviews Dataset from Kaggle into a DataFrame.

Python

```
#Narmadha K
#241801181

!pip install nltk pandas kaggle -q

import pandas as pd
import nltk
import re
import string

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag, ne_chunk

from google.colab import files
uploaded = files.upload()

df = pd.read_csv('Reviews.csv')

print(df.head())
```

2. Install and Import Required Libraries.

Python

```
#Narmadha K
#241801181
```

```
!pip install nltk pandas -q

import pandas as pd
import nltk
import re
import string

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk import pos_tag, ne_chunk

print("Required libraries imported successfully")
```

3. Import NLTK Resources.

```
Python
#Narmadha K
#241801181

nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('maxent_ne_chunker')
nltk.download('maxent_ne_chunker_tab')
nltk.download('words')

print("NLTK resources downloaded successfully")
```

4. Load the Dataset.

a. Select the column containing review text.

b. Remove any missing or null entries.

c. Limit the dataset to 10,000 reviews for faster processing.

Python

```python
#Narmadha K
#241801181

df = pd.read_csv('Reviews.csv')

df = df[['Text']]

df = df.dropna(subset=['Text'])

df = df.head(10000)

print("Dataset loaded successfully")
print("Number of reviews:", len(df))
print(df.head())
```

5. Perform the Preprocessing Step.

a. Lowercase.

b. Remove punctuation.

Python

```python
#Narmadha K
#241801181

def preprocess_text(text):
    text = text.lower()
    text = text.translate(
        str.maketrans('', '', string.punctuation)
    )
    return text

df['Cleaned_Text'] = df['Text'].apply(preprocess_text)

print(df[['Text', 'Cleaned_Text']].head())
```

6. Perform Tokenization and Remove Stopwords, Non-Alphabetic Tokens.

Python

```
#Narmadha K
#241801181

stop_words = set(stopwords.words('english'))

def tokenize_text(text):
    tokens = word_tokenize(text)

    filtered_tokens = [
        word for word in tokens
        if word.isalpha() and word not in stop_words
    ]

    return filtered_tokens

df['Tokens'] = df['Cleaned_Text'].apply(tokenize_text)

print("Tokenized reviews:")
print(df['Tokens'].head())
```

7. Perform Parts of Speech (POS) Tagging.

Python

```
#Narmadha K
#241801181

df['POS_Tagging'] = df['Tokens'].apply(pos_tag)

print("Parts of Speech Tagging:")
print(df['POS_Tagging'].head())
```

8. Identify Named Entity Recognition (NER) for the Separated Tokens.

Python

```
#Narmadha K
#241801181
```

```
df['NER'] = df['POS_Tagging'].apply(ne_chunk)

def extract_entities(tree):
    entities = []

    for subtree in tree:
        if hasattr(subtree, 'label'):
            entity_name = ' '.join(
                word for word, tag in subtree.leaves()
            )
            entities.append((entity_name, subtree.label()))

    return entities

df['Named_Entities'] = df['NER'].apply(extract_entities)

print("Named Entity Recognition:")
print(df[['Tokens', 'POS_Tagging', 'Named_Entities']].head())
```

RESULT

Thus, information extraction was performed successfully on the Amazon Fine Food Reviews dataset using NLTK. The reviews were preprocessed, tokenized, and filtered to remove stopwords and non-alphabetic tokens. Parts of Speech tagging and Named Entity Recognition were also performed successfully.

Ex5b: INFORMATION EXTRACTION USING spaCy

1. Download Amazon Fine Food Reviews Dataset from Kaggle into a DataFrame.

Python

```
#Narmadha K
#241801181

!pip install pandas spacy -q

import pandas as pd
import spacy
import string
import re

from google.colab import files
uploaded = files.upload()

df = pd.read_csv('Reviews.csv')

print(df.head())
```

2. Install and Import Required Libraries.

Python

```
#Narmadha K
#241801181

!pip install pandas spacy -q

import pandas as pd
import spacy
import string
```

```python
import re

print("Required libraries imported successfully")
```

3. Import NLTK Resources.

```python
#Narmadha K
#241801181

import nltk

nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')

print("NLTK resources downloaded successfully")
```

4. Load the Dataset.

a. Select the column containing review text.

b. Remove any missing or null entries.

c. Limit the dataset to 10,000 reviews for faster processing.

```python
#Narmadha K
#241801181

df = pd.read_csv('Reviews.csv')

df = df[['Text']]

df = df.dropna(subset=['Text'])

df = df.head(10000)
```

```
print("Dataset loaded successfully")
print("Number of reviews:", len(df))
print(df.head())
```

5. Do the Preprocessing Step.

a. Lowercase.

b. Remove punctuation.

Python

```
#Narmadha K
#241801181

def preprocess_text(text):
    text = text.lower()
    text = text.translate(
        str.maketrans('', '', string.punctuation)
    )
    return text

df['Cleaned_Text'] = df['Text'].apply(preprocess_text)

print(df[['Text', 'Cleaned_Text']].head())
```

6. Perform Tokenization and Remove Stopwords, Non-Alphabetic Tokens.

Python

```
#Narmadha K
#241801181

nltk.download('stopwords')

from nltk.corpus import stopwords
```

```python
stop_words = set(stopwords.words('english'))

def tokenize_text(text):
    tokens = text.split()

    filtered_tokens = [
        word for word in tokens
        if word.isalpha() and word not in stop_words
    ]

    return filtered_tokens

df['Tokens'] = df['Cleaned_Text'].apply(tokenize_text)

print("Tokenized reviews:")
print(df['Tokens'].head())
```

7. Perform Parts of Speech (POS) Tagging.

Python

```python
#Narmadha K
#241801181

nlp = spacy.load("en_core_web_sm")

def pos_tagging(tokens):
    text = " ".join(tokens)
    doc = nlp(text)

    return [(token.text, token.pos_) for token in doc]

df['POS_Tagging'] = df['Tokens'].apply(pos_tagging)

print("Parts of Speech Tagging:")
print(df['POS_Tagging'].head())
```

8. Identify Named Entity Recognition (NER) for the Separated Tokens.

Python

```
#Narmadha K
#241801181

def extract_ner(tokens):
    text = " ".join(tokens)
    doc = nlp(text)

    entities = [
        (ent.text, ent.label_)
        for ent in doc.ents
    ]

    return entities

df['Named_Entities'] = df['Tokens'].apply(extract_ner)

print("Named Entity Recognition:")
print(df[['Tokens', 'POS_Tagging', 'Named_Entities']].head())
```

RESULT

Thus, information extraction was performed successfully on the Amazon Fine Food Reviews dataset using spaCy. The reviews were loaded and preprocessed by converting the text to lowercase and removing punctuation. Tokenization, stopword removal, and filtering of non-alphabetic tokens were performed. Parts of Speech tagging and Named Entity Recognition were also carried out using spaCy.

Ex6a: EXPLORATORY DATA ANALYSIS (EDA)

Learning Objective:

The objective of this experiment is to enable students to understand and perform Exploratory Data Analysis (EDA) on a real-world Netflix Titles dataset using Python.

Problem Statement:

Netflix maintains a large collection of movies and TV shows containing information such as title, content type, release year, date added, country, rating, duration, genre, director, cast, and description. The objective is to analyse the Netflix content library and identify important patterns using statistical techniques and visualisations.

1. Download Netflix Titles Dataset from Kaggle and Convert It to a DataFrame.

Python

```
#Narmadha K
#241801181

!pip install pandas numpy matplotlib seaborn wordcloud -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from wordcloud import WordCloud
import re
from google.colab import files

uploaded = files.upload()

df = pd.read_csv('netflix_titles.csv')
```

```
print("Dataset loaded successfully")
print(df.head())
```

2. Explore the Dataset.

Python

```
#Narmadha K
#241801181

print("First five rows:")
print(df.head())

print("\nLast five rows:")
print(df.tail())

print("\nDataset shape:")
print(df.shape)

print("\nColumn names:")
print(df.columns)

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
print(df.describe(include='all'))

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate rows:")
print(df.duplicated().sum())
```

3. Content Distribution Analysis.

a. How many Movies vs TV Shows are in the dataset?

Python

```
#Narmadha K
#241801181

content_counts = df['type'].value_counts()

print(content_counts)

plt.figure(figsize=(7, 5))
sns.barplot(x=content_counts.index, y=content_counts.values)
plt.title('Movies vs TV Shows on Netflix')
plt.xlabel('Content Type')
plt.ylabel('Number of Titles')
plt.show()

plt.figure(figsize=(6, 6))
plt.pie(content_counts.values, labels=content_counts.index,
        autopct='%1.1f%%', startangle=90)
plt.title('Content Type Distribution')
plt.show()
```

b. What is the distribution of ratings (e.g., TV-MA, PG, R, etc.)?

Python

```
#Narmadha K
#241801181

rating_counts = df['rating'].value_counts()

print(rating_counts)

plt.figure(figsize=(12, 6))
sns.barplot(x=rating_counts.index, y=rating_counts.values)
plt.title('Distribution of Netflix Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of Titles')
```

```
plt.xticks(rotation=45)
plt.show()
```

c. Which countries have the most Netflix content?

Python

```
#Narmadha K
#241801181

countries = df['country'].dropna().str.split(', ').explode()
top_countries = countries.value_counts().head(10)

print(top_countries)

plt.figure(figsize=(12, 6))
sns.barplot(x=top_countries.values, y=top_countries.index)
plt.title('Top 10 Countries Producing Netflix Content')
plt.xlabel('Number of Titles')
plt.ylabel('Country')
plt.show()
```

4. Time-Based Trend Analysis.

a. In which years were the most shows/movies released?

Python

```
#Narmadha K
#241801181

release_counts = df['release_year'].value_counts().sort_index()

print("Top release years:")
print(df['release_year'].value_counts().head(10))

plt.figure(figsize=(14, 6))
release_counts.plot(kind='line', marker='o')
```

```
plt.title('Netflix Titles by Release Year')
plt.xlabel('Release Year')
plt.ylabel('Number of Titles')
plt.show()
```

b. In which years were titles added to Netflix?

Python

```
#Narmadha K
#241801181

df['date_added'] = pd.to_datetime(
    df['date_added'].str.strip(), errors='coerce'
)

df['year_added'] = df['date_added'].dt.year

added_counts = df['year_added'].value_counts().sort_index()

print("Titles added by year:")
print(added_counts)

plt.figure(figsize=(12, 6))
sns.barplot(x=added_counts.index, y=added_counts.values)
plt.title('Netflix Titles Added by Year')
plt.xlabel('Year Added')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.show()
```

c. How has Netflix's content growth trend changed over time?

Python

```
#Narmadha K
#241801181
```

```
growth = df.groupby('year_added').size().sort_index()

print(growth)

plt.figure(figsize=(12, 6))
plt.plot(growth.index, growth.values, marker='o')
plt.title('Netflix Content Growth Trend')
plt.xlabel('Year Added')
plt.ylabel('Number of Titles Added')
plt.grid(True)
plt.show()
```

5. Duration and Content Type Analysis.

a. What is the distribution of movie durations (in minutes)?

Python

```
#Narmadha K
#241801181

movies = df[df['type'] == 'Movie'].copy()

movies['duration_minutes'] = pd.to_numeric(
    movies['duration'].str.extract(r'(\d+)')[0],
    errors='coerce'
)

print(movies['duration_minutes'].describe())

plt.figure(figsize=(10, 6))
sns.histplot(movies['duration_minutes'].dropna(),
             bins=30, kde=True)
plt.title('Distribution of Movie Durations')
plt.xlabel('Duration (Minutes)')
plt.ylabel('Number of Movies')
plt.show()
```

b. How many TV shows exist per number of seasons?

Python

```
#Narmadha K
#241801181

tv_shows = df[df['type'] == 'TV Show'].copy()

tv_shows['seasons'] = pd.to_numeric(
    tv_shows['duration'].str.extract(r'(\d+)')[0],
    errors='coerce'
)

season_counts = tv_shows['seasons'].value_counts().sort_index()

print(season_counts)

plt.figure(figsize=(10, 6))
sns.barplot(x=season_counts.index, y=season_counts.values)
plt.title('Number of TV Shows by Seasons')
plt.xlabel('Number of Seasons')
plt.ylabel('Number of TV Shows')
plt.show()
```

6. Genre and Categories Analysis.

a. What are the top 10 most common genres on Netflix?

Python

```
#Narmadha K
#241801181

genres = df['listed_in'].dropna().str.split(', ').explode()
top_genres = genres.value_counts().head(10)

print(top_genres)

plt.figure(figsize=(12, 6))
```

```
sns.barplot(x=top_genres.values, y=top_genres.index)
plt.title('Top 10 Netflix Genres')
plt.xlabel('Number of Titles')
plt.ylabel('Genre')
plt.show()
```

b. Which directors have the most titles on Netflix?

Python

```
#Narmadha K
#241801181

directors = df['director'].dropna().str.split(', ').explode()
top_directors = directors.value_counts().head(10)

print(top_directors)

plt.figure(figsize=(12, 6))
sns.barplot(x=top_directors.values, y=top_directors.index)
plt.title('Top 10 Directors on Netflix')
plt.xlabel('Number of Titles')
plt.ylabel('Director')
plt.show()
```

c. Which actors appear most frequently across Netflix shows/movies?

Python

```
#Narmadha K
#241801181

actors = df['cast'].dropna().str.split(', ').explode()
top_actors = actors.value_counts().head(10)

print(top_actors)

plt.figure(figsize=(12, 6))
```

```
sns.barplot(x=top_actors.values, y=top_actors.index)
plt.title('Top 10 Actors on Netflix')
plt.xlabel('Number of Appearances')
plt.ylabel('Actor')
plt.show()
```

7. Text and Descriptions.

a. What are the most common words in the description field? (Word Cloud)

Python

```
#Narmadha K
#241801181

text = ' '.join(df['description'].dropna().astype(str))

text = re.sub(r'[^a-zA-Z\s]', '', text).lower()

wordcloud = WordCloud(
    width=1000,
    height=500,
    background_color='white',
    stopwords=WordCloud().stopwords,
    max_words=100
).generate(text)

plt.figure(figsize=(14, 7))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Word Cloud of Netflix Descriptions')
plt.show()

from collections import Counter

words = text.split()
common_words = Counter(words).most_common(20)
```

```
print("Most common words:")
print(common_words)
```

8. In Which Months of the Year Does Netflix Add the Most Content?

Python

```
#Narmadha K
#241801181

df['month_added'] = df['date_added'].dt.month

month_counts = df['month_added'].value_counts().sort_index()

month_names = [
    'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
    'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'
]

month_counts.index = [
    month_names[m - 1] for m in month_counts.index
]

print(month_counts.sort_values(ascending=False))

plt.figure(figsize=(12, 6))
sns.barplot(x=month_counts.index, y=month_counts.values)
plt.title('Netflix Content Added by Month')
plt.xlabel('Month')
plt.ylabel('Number of Titles')
plt.show()
```

9. Which Countries Produce More Movies vs TV Shows?

Python

```python
#Narmadha K
#241801181

country_data = df.dropna(subset=['country']).copy()

country_data['country'] = country_data['country'].str.split(', ')
country_data = country_data.explode('country')

country_type = pd.crosstab(
    country_data['country'],
    country_data['type']
)

top_country_names =
country_data['country'].value_counts().head(10).index

country_type_top = country_type.loc[
    country_type.index.intersection(top_country_names)
]

print(country_type_top)

country_type_top.plot(kind='bar', figsize=(12, 6))
plt.title('Movies vs TV Shows by Country')
plt.xlabel('Country')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.legend(title='Content Type')
plt.tight_layout()
plt.show()
```

10. What Are the Most Common Ratings for Movies vs TV Shows?

Python

```python
#Narmadha K
#241801181

rating_type = pd.crosstab(df['rating'], df['type'])

print(rating_type)

rating_type.plot(kind='bar', figsize=(14, 6))
plt.title('Ratings for Movies vs TV Shows')
plt.xlabel('Rating')
plt.ylabel('Number of Titles')
plt.xticks(rotation=45)
plt.legend(title='Content Type')
plt.tight_layout()
plt.show()
```

11. What Are the Oldest and Newest Titles Available on Netflix?

Python

```python
#Narmadha K
#241801181

oldest_title = df.loc[df['release_year'].idxmin()]
newest_title = df.loc[df['release_year'].idxmax()]

print("Oldest title:")
print(oldest_title[['title', 'type', 'release_year']])

print("\nNewest title:")
print(newest_title[['title', 'type', 'release_year']])

print("\nOldest release year:", df['release_year'].min())
print("Newest release year:", df['release_year'].max())
```

12. Which Genres Frequently Co-Occur Together? (Genre Correlation Heatmap)

Python

```python
#Narmadha K
#241801181

from itertools import combinations
from collections import Counter

genre_lists = (
    df['listed_in']
    .dropna()
    .apply(lambda x: [g.strip() for g in x.split(',')])
)

genre_pairs = Counter()

for genre_list in genre_lists:
    unique_genres = sorted(set(genre_list))

    for pair in combinations(unique_genres, 2):
        genre_pairs[pair] += 1

top_genres = [
    genre for genre, count in
    Counter(g for pair, count in genre_pairs.items()
            for g in pair).most_common(10)
]

cooccurrence = pd.DataFrame(
    0, index=top_genres, columns=top_genres
)

for (g1, g2), count in genre_pairs.items():
    if g1 in top_genres and g2 in top_genres:
        cooccurrence.loc[g1, g2] = count
```

```
        cooccurrence.loc[g2, g1] = count

plt.figure(figsize=(12, 8))
sns.heatmap(cooccurrence, annot=True, fmt='g', cmap='YlGnBu')
plt.title('Genre Co-occurrence Heatmap')
plt.xlabel('Genre')
plt.ylabel('Genre')
plt.show()

print("Genre co-occurrence matrix:")
print(cooccurrence)
```

RESULT

Thus, Exploratory Data Analysis was performed successfully on the Netflix Titles dataset using Python. The dataset was explored and analysed to identify content distribution, ratings, country-wise content, release and addition trends, movie durations, TV show seasons, popular genres, directors, actors, description keywords, monthly additions, and genre co-occurrence patterns using suitable statistical methods and visualisations.

EX6(b): TIME SERIES ANALYSIS AND FORECASTING USING ARIMA

Learning Objective

The objective of this experiment is to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement

An airline company maintains monthly records of the number of passengers travelling by air. The company wants to understand historical passenger trends and seasonal patterns and use this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, check stationarity, apply suitable transformations, build an ARIMA forecasting model, predict passenger numbers, and evaluate the accuracy of the forecasts.

1. Download Airline Passengers Dataset from GitHub.

Python

```
#Narmadha K

#241801181


!pip install pandas numpy matplotlib statsmodels scikit-learn -q


import pandas as pd

import numpy as np

import matplotlib.pyplot as plt


url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/airlin
e-passengers.csv"


df = pd.read_csv(url)


print("Dataset downloaded successfully")

print(df.head())
```

2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.

Python

```
#Narmadha K
#241801181


df = pd.read_csv(
    url,
    parse_dates=['Month'],
    index_col='Month'
)


df.index = pd.DatetimeIndex(df.index)


print(df.head())
print("\nIndex type:", type(df.index))
print("\nDataset information:")
print(df.info())
```

3. Explore the Dataset.

Python

```
#Narmadha K
#241801181


print("First five rows:")
```

```
print(df.head())


print("\nLast five rows:")
print(df.tail())


print("\nDataset shape:")
print(df.shape)


print("\nColumn names:")
print(df.columns)


print("\nStatistical summary:")
print(df.describe())


print("\nMissing values:")
print(df.isnull().sum())
```

4. Plot the Series to Inspect Trend, Seasonality, and Outliers.

Python

```
#Narmadha K

#241801181
```

```
plt.figure(figsize=(12, 5))


plt.plot(df['Passengers'], marker='o')


plt.title('Monthly Airline Passenger Data')
plt.xlabel('Year')
plt.ylabel('Number of Passengers')
plt.grid(True)


plt.show()


from statsmodels.tsa.seasonal import seasonal_decompose


decomposition = seasonal_decompose(
    df['Passengers'],
    model='multiplicative',
    period=12
)


decomposition.plot()
plt.show()
```

5. Do Stationarity Check by Running Augmented Dickey-Fuller (ADF) Test.

Python

```
#Narmadha K

#241801181


from statsmodels.tsa.stattools import adfuller


result = adfuller(df['Passengers'])


print("ADF Statistic:", result[0])

print("p-value:", result[1])


if result[1] <= 0.05:

    print("The series is stationary")

else:

    print("The series is non-stationary")
```

6. If the Series Is Non-Stationary, Convert It to Stationary by Differencing the Series.

Python

```
#Narmadha K

#241801181
```

```
df['First_Difference'] = df['Passengers'].diff()


differenced_series = df['First_Difference'].dropna()


plt.figure(figsize=(12, 5))
plt.plot(differenced_series)


plt.title('First-Order Differenced Series')
plt.xlabel('Year')
plt.ylabel('Differenced Passenger Values')
plt.grid(True)


plt.show()
```

7. Re-run ADF on the Differenced Series and Visualize the New DataFrame.

Python

```
#Narmadha K
#241801181


result_diff = adfuller(differenced_series)


print("ADF Statistic:", result_diff[0])
```

```
print("p-value:", result_diff[1])


if result_diff[1] <= 0.05:
    print("The differenced series is stationary")
else:
    print("The differenced series is non-stationary")


print("\nDifferenced DataFrame:")
print(df[['Passengers', 'First_Difference']].head(15))
```

8. If Still Non-Stationary, Consider Another Difference (d=2) or Log Transformation Before Differencing.

Python

```
#Narmadha K
#241801181


d = 1


if result_diff[1] > 0.05:
    log_series = np.log(df['Passengers'])


    log_diff = log_series.diff().dropna()
```

```
    result_log = adfuller(log_diff)


    print("Log-differenced ADF Statistic:", result_log[0])

    print("Log-differenced p-value:", result_log[1])


    if result_log[1] <= 0.05:

        d = 1

        print("Log transformation followed by first differencing is
stationary")

    else:

        second_diff = differenced_series.diff().dropna()

        result_second = adfuller(second_diff)


        print("Second-difference ADF Statistic:", result_second[0])

        print("Second-difference p-value:", result_second[1])


        if result_second[1] <= 0.05:

            d = 2

            print("Second differencing achieved stationarity")

        else:

            d = 2

            print("Series may need further analysis; using d=2")
```

```
else:
    print("First differencing achieved stationarity")

print("Selected differencing order d =", d)
```

9. Split the Dataset for Training and Testing on the Original Series.

Python

```
#Narmadha K
#241801181

series = df['Passengers']

train_size = int(len(series) * 0.8)

train = series.iloc[:train_size]
test = series.iloc[train_size:]

print("Training data size:", len(train))
print("Testing data size:", len(test))

print("\nTraining data:")
```

```
print(train.tail())


print("\nTesting data:")

print(test.head())
```

10. Fit ARIMA Model on Training Set. The d Value Comes from Step 8.

Python

```
#Narmadha K

#241801181


from statsmodels.tsa.arima.model import ARIMA


model = ARIMA(

    train,

    order=(2, d, 2)

)


model_fit = model.fit()


print(model_fit.summary())
```

11. Perform Forecasts on the Test Dataset.

Python

```python
#Narmadha K

#241801181


forecast = model_fit.forecast(steps=len(test))


forecast = pd.Series(

    forecast,

    index=test.index,

    name='Predicted_Passengers'

)


print("Forecasted Passenger Numbers:")

print(forecast)


comparison = pd.DataFrame({

    'Actual': test,

    'Predicted': forecast

})


print("\nActual vs Predicted:")

print(comparison)
```

12. Compute Metrics MSE, MAE, and RMSE on the Predictions.

Python

```
#Narmadha K

#241801181


from sklearn.metrics import mean_squared_error, mean_absolute_error


mse = mean_squared_error(test, forecast)

mae = mean_absolute_error(test, forecast)

rmse = np.sqrt(mse)


print("Mean Squared Error (MSE):", mse)

print("Mean Absolute Error (MAE):", mae)

print("Root Mean Squared Error (RMSE):", rmse)
```

13. Plot Train, Test, and Predictions in a Single Plot.

Python

```
#Narmadha K

#241801181


plt.figure(figsize=(12, 6))


plt.plot(train.index, train,
```

```
         label='Training Data')


plt.plot(test.index, test,
         label='Actual Test Data')


plt.plot(forecast.index, forecast,
         label='ARIMA Forecast',
         linestyle='--')


plt.title('Airline Passenger Forecast using ARIMA')
plt.xlabel('Year')
plt.ylabel('Number of Passengers')


plt.legend()
plt.grid(True)
plt.show()
```

RESULT

Thus, time series analysis and forecasting were performed successfully on the Airline Passengers dataset using Python. The time series was visualised, stationarity was checked using the Augmented Dickey-Fuller test, and differencing was applied as required. An ARIMA model was fitted to the training data, passenger numbers were forecast for the test period, and the prediction accuracy was evaluated using MSE, MAE, and RMSE. The training data, actual test values, and predicted values were visualised in a single plot.

Ex7: MODEL PLANNING AND BUILDING – CLUSTERING

Learning Objective

The objective of this experiment is to understand and implement the K-Means clustering algorithm for customer segmentation.

Problem Statement

A retail company wants to group its customers based on their Annual Income and Spending Score to design personalized marketing campaigns. The K-Means clustering algorithm is used to identify customer segments, visualize the clusters, and recommend suitable marketing strategies.

1. Use Mall Customer Segmentation Dataset from Kaggle and Load It into a Pandas DataFrame.

None

Narmadha K
241801181

```
!pip install pandas numpy matplotlib seaborn scikit-learn -q

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
uploaded = files.upload()

df = pd.read_csv('Mall_Customers.csv')

print("Dataset loaded successfully")
print(df.head())
```

2. Explore the Dataset.

Display the first and last five records, check the dataset shape and data types, identify missing values and duplicate records, and display summary statistics.

None

Narmadha K
241801181

```
print("First five records:")
print(df.head())

print("\nLast five records:")
print(df.tail())

print("\nDataset shape:")
print(df.shape)

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate records:")
print(df.duplicated().sum())

print("\nSummary statistics:")
print(df.describe(include='all'))
```

3. Select the Important Features for Clustering: Annual Income (k$), Spending Score (1–100).

None

Narmadha K
241801181

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
```

```
print("Selected features:")
print(X.head())

print("\nFeature information:")
print(X.info())
```

4. Standardize the Selected Features Using StandardScaler for Clustering.

None

Narmadha K
241801181

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(
    X_scaled,
    columns=['Annual Income (Scaled)', 'Spending Score (Scaled)']
)

print("Standardized features:")
print(X_scaled_df.head())
```

5. Visualize the Selected Features Using Scatter Plot and Pair Plot.

a. Scatter Plot

None

Narmadha K
241801181

```
plt.figure(figsize=(8, 6))
```

```
plt.scatter(
    X['Annual Income (k$)'],
    X['Spending Score (1-100)']
)

plt.title('Annual Income vs Spending Score')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.grid(True)

plt.show()
```

b. Pair Plot

None

Narmadha K
241801181

```
sns.pairplot(
    df[['Annual Income (k$)', 'Spending Score (1-100)']]
)

plt.suptitle('Pair Plot of Customer Features', y=1.02)
plt.show()
```

MODEL PLANNING AND BUILDING

6. Determine the Optimal Number of Clusters Using the Elbow Method. Plot the Elbow Curve and Identify the Optimal Value of K.

None

Narmadha K
241801181

```
from sklearn.cluster import KMeans

wcss = []

for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        init='k-means++',
        random_state=42,
        n_init=10
    )

    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 5))

plt.plot(range(1, 11), wcss, marker='o')

plt.title('Elbow Method for Optimal K')
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.xticks(range(1, 11))
plt.grid(True)

plt.show()
```

Observation: The optimal number of clusters is selected by identifying the elbow point, where the decrease in WCSS starts slowing down. For this dataset, (K=5) is a commonly used choice.

7. Train the K-Means Clustering Model Using the Optimal Number of Clusters. Assign Cluster Labels to Each Customer, Add Them as a New Column, and Save the Dataset as a File.

None

Narmadha K
241801181

```
optimal_k = 5

kmeans = KMeans(
    n_clusters=optimal_k,
    init='k-means++',
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)

print("Cluster labels assigned successfully")
print(df.head())

df.to_csv('Mall_Customers_Clustered.csv', index=False)

print("\nClustered dataset saved successfully")

from google.colab import files
files.download('Mall_Customers_Clustered.csv')
```

CLUSTER VISUALIZATION

8. Visualize the Customer Clusters Using a Scatter Plot. Use Annual Income on the X-axis, Spending Score on the Y-axis, Different Colors for Each Cluster, Highlight Cluster Centroids, and Display Their Coordinates.

None

Narmadha K
241801181

```
centroids_scaled = kmeans.cluster_centers_

centroids_original = scaler.inverse_transform(centroids_scaled)
```

```
centroids_df = pd.DataFrame(
    centroids_original,
    columns=['Annual Income (k$)', 'Spending Score (1-100)']
)

plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=df,
    x='Annual Income (k$)',
    y='Spending Score (1-100)',
    hue='Cluster',
    palette='tab10',
    s=80
)

plt.scatter(
    centroids_df['Annual Income (k$)'],
    centroids_df['Spending Score (1-100)'],
    marker='X',
    s=250,
    color='black',
    label='Centroids'
)

for i, row in centroids_df.iterrows():
    plt.annotate(
        f"C{i}: ({row.iloc[0]:.1f}, {row.iloc[1]:.1f})",
        (row.iloc[0], row.iloc[1]),
        xytext=(5, 5),
        textcoords='offset points'
    )

plt.title('Customer Segmentation Using K-Means')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
```

```
plt.legend()
plt.grid(True)
plt.show()

print("Cluster centroid coordinates:")
print(centroids_df.round(2))
```

CLUSTER ANALYSIS

9. Calculate the Number of Customers in Each Cluster.

None

Narmadha K

241801181

```
cluster_counts = df['Cluster'].value_counts().sort_index()

print("Number of customers in each cluster:")
print(cluster_counts)

plt.figure(figsize=(8, 5))

sns.barplot(
    x=cluster_counts.index,
    y=cluster_counts.values
)

plt.title('Number of Customers in Each Cluster')
plt.xlabel('Cluster')
plt.ylabel('Number of Customers')
plt.show()
```

10. Compute the Average Annual Income and Spending Score for Each Cluster.

None

Narmadha K
241801181

```
cluster_analysis = df.groupby('Cluster').agg(
    Customer_Count=('CustomerID', 'count'),
    Average_Annual_Income=('Annual Income (k$)', 'mean'),
    Average_Spending_Score=('Spending Score (1-100)', 'mean')
).round(2)

print("Cluster Analysis:")
print(cluster_analysis)
```

MARKETING STRATEGIES

None

Narmadha K
241801181

```
for cluster, row in cluster_analysis.iterrows():
    income = row['Average_Annual_Income']
    spending = row['Average_Spending_Score']

    print(f"\nCluster {cluster}:")
    print(f"Average Annual Income: {income}")
    print(f"Average Spending Score: {spending}")

    if income >= 60 and spending >= 60:
        print("Strategy: Offer loyalty rewards and premium
products.")
    elif income >= 60 and spending < 40:
        print("Strategy: Provide targeted discounts and exclusive
offers.")
    elif income < 40 and spending >= 60:
        print("Strategy: Offer affordable premium deals and reward
points.")
    elif income < 40 and spending < 40:
```

```
        print("Strategy: Use budget-friendly promotions to increase
engagement.")
    else:
        print("Strategy: Recommend personalized offers and
cross-selling.")
```

RESULT

Thus, customer segmentation was performed successfully using the K-Means clustering algorithm. The Mall Customer Segmentation dataset was explored, the Annual Income and Spending Score features were standardized, and the optimal number of clusters was investigated using the Elbow Method. Customers were grouped into clusters, the cluster centroids were visualized, and the number of customers and average feature values were calculated for each cluster. Suitable marketing strategies were recommended based on the identified customer segments.

EX8: INTERACTIVE DASHBOARDS WITH TABLEAU

Learning Objective

The objective of this experiment is to understand and implement interactive data visualisation and dashboard design using Tableau.

Problem Statement

Create an interactive dashboard in Tableau using the Sample – Superstore dataset from Kaggle. The dashboard should include a bar chart showing Sales by Category, a line chart showing Sales trends over time, a scatter plot showing Profit vs Sales by Sub-Category, and interactive filters and dashboard actions.

1. Open Tableau and Load Dataset

Procedure:

1. Open Tableau Desktop or Tableau Public.
2. Download the Sample Superstore dataset from Kaggle.
3. Select **Connect** → **To a File** → **Microsoft Excel** or **Text File**, depending on the downloaded file format.
4. Select the downloaded dataset file.
5. Check the data preview and click **Sheet 1** to begin creating visualisations.

Dataset: Sample Superstore

Important fields: Order Date, Category, Sub-Category, Region, Sales, Profit, Segment, and Quantity.

2. Explore the Data

Procedure:

1. Check the field names and data types in the Data pane.
2. Ensure that Order Date is recognised as a Date field.
3. Ensure that Sales, Profit, and Quantity are recognised as numerical measures.

4. Check for missing values and verify that Category, Sub-Category, and Region are available.
5. Drag Sales and Profit into a worksheet to explore their aggregated values.

Expected Output:

The dataset is loaded successfully, and the required dimensions and measures are available for creating charts.

3. Create Worksheet 1 – Bar Chart (Sales by Category)

Procedure:

1. Click the Sheet 1 tab and rename it **Bar_Sales_by_Category**.
2. Drag **Category** to Columns.
3. Drag **Sales** to Rows. Ensure the aggregation is SUM(Sales).
4. Sort the bars in descending order using the sort icon.
5. Drag **Region** to Color in the Marks card.
6. Click Label and select **Show Mark Labels**.
7. Optionally, drag Region to Filters, select all regions, and choose **Show Filter**.

Expected Output:

A bar chart displaying total sales for Furniture, Office Supplies, and Technology. Different colours represent the regions, and the bar labels display the sales values.

4. Create Worksheet 2 – Line Chart (Sales Trend Over Time)

Procedure:

1. Click New Worksheet and rename it **Line_Sales_Trend**.
2. Drag **Order Date** to Columns.
3. Open the Order Date pill menu and select **Month (continuous)**, displayed as a green pill.
4. Drag **Sales** to Rows, ensuring SUM(Sales).
5. Drag **Region** to Color to create separate lines for each region.
6. Right-click the vertical axis, select Format, and format the values as currency if required.
7. Optionally, add a Region or Category filter and select Show Filter.

Expected Output:

A line chart showing monthly sales trends over time, with separate coloured lines for each region. The chart helps identify sales growth, declines, and seasonal patterns.

5. Create Worksheet 3 – Scatter Plot (Profit vs Sales)

Procedure:

1. Click New Worksheet and rename it **Scatter_Profit_vs_Sales**.
2. Drag **Sales** to Columns and ensure SUM(Sales).
3. Drag **Profit** to Rows and ensure SUM(Profit).
4. Drag **Sub-Category** to Detail on the Marks card so that each point represents a sub-category.
5. Drag **Category** to Color.
6. Optionally, drag Segment to Shape.
7. Open the Analytics pane and drag an **Average Line** onto the Table for the horizontal and vertical axes, where supported.

Expected Output:

A scatter plot displaying the relationship between Sales and Profit for different sub-categories. Points above the zero-profit level represent profitable sub-categories, while points below it represent loss-making sub-categories.

6. Build the Dashboard

Procedure:

1. Click **New Dashboard** at the bottom of the Tableau window.
2. Rename the dashboard **Sales Dashboard**.
3. Set the dashboard size to approximately **1200 × 800 pixels**.
4. From the Sheets pane, drag the three worksheets onto the dashboard.
5. Place the Line_Sales_Trend chart at the top.
6. Place Bar_Sales_by_Category at the bottom-left.
7. Place Scatter_Profit_vs_Sales at the bottom-right.
8. Use horizontal and vertical layout containers to align the charts.
9. Add a dashboard title: **Sales and Profit Dashboard**.
10. Adjust chart sizes, legends, labels, and spacing for readability.

Expected Output:

A single dashboard containing the sales trend line chart, category sales bar chart, and profit-versus-sales scatter plot in a well-organised layout.

7. Add Interactivity

a. Add Filters

Procedure:

1. Open the dashboard.
2. Click the dropdown arrow at the top-right of a worksheet.
3. Select **Filters** → **Region** or choose Show Filter for an available field.
4. Repeat for Category or Order Date, as required.
5. Configure the filter card to apply to the relevant worksheets using **Apply to Worksheets** → **Selected Worksheets** or **All Using This Data Source**.

Expected Output:

Users can select a region, category, or date range to filter the dashboard and analyse specific parts of the dataset.

b. Add Dashboard Filter Action

Procedure:

1. Select **Dashboard** → **Actions** → **Add Action** → **Filter**.
2. Set the source sheet to **Bar_Sales_by_Category**.
3. Select **Line_Sales_Trend** and **Scatter_Profit_vs_Sales** as target sheets.
4. Set Run action on to **Select**.
5. Set Clearing the selection to **Show all values**.
6. Click OK to save the action.

Expected Output:

Selecting a category or bar in the bar chart filters the target charts according to the selected data. Clear the selection to restore all values.

c. Add Highlight Action (Optional)

Procedure:

1. Select **Dashboard** → **Actions** → **Add Action** → **Highlight**.
2. Select the source and target worksheets.
3. Choose Sub-Category or Category as the highlighting field.
4. Configure the action to run on Hover.
5. Click OK.

Expected Output:

Hovering over a relevant data mark highlights matching data in other charts, making it easier to compare related information.

8. Publish or Export

Procedure:

Option A – Publish to Tableau Public

1. Save the workbook.
2. Select the Tableau Public publishing option available in your version of Tableau.
3. Sign in to your Tableau Public account.
4. Enter the workbook name and publish it.
5. Open the published dashboard and verify that the filters and actions work.

Option B – Export as PDF or Image

1. Open the completed dashboard.
2. Select **File → Export as PDF** or the appropriate image-export option available in your Tableau version.
3. Choose the dashboard and required layout settings.
4. Save the exported file.
5. Open the exported file and verify that the charts and dashboard title are visible.

Expected Output:

The completed dashboard is published online or exported as a PDF/image for documentation and presentation.

OUTPUT

The following visualisations are obtained after completing the experiment:

Output 1 – Bar Chart: Sales by Category

A bar chart showing total sales for Furniture, Office Supplies, and Technology, with regional colour coding and sales labels.

Output 2 – Line Chart: Sales Trend Over Time

A line chart showing monthly sales trends, with separate lines for different regions.

Output 3 – Scatter Plot: Profit vs Sales

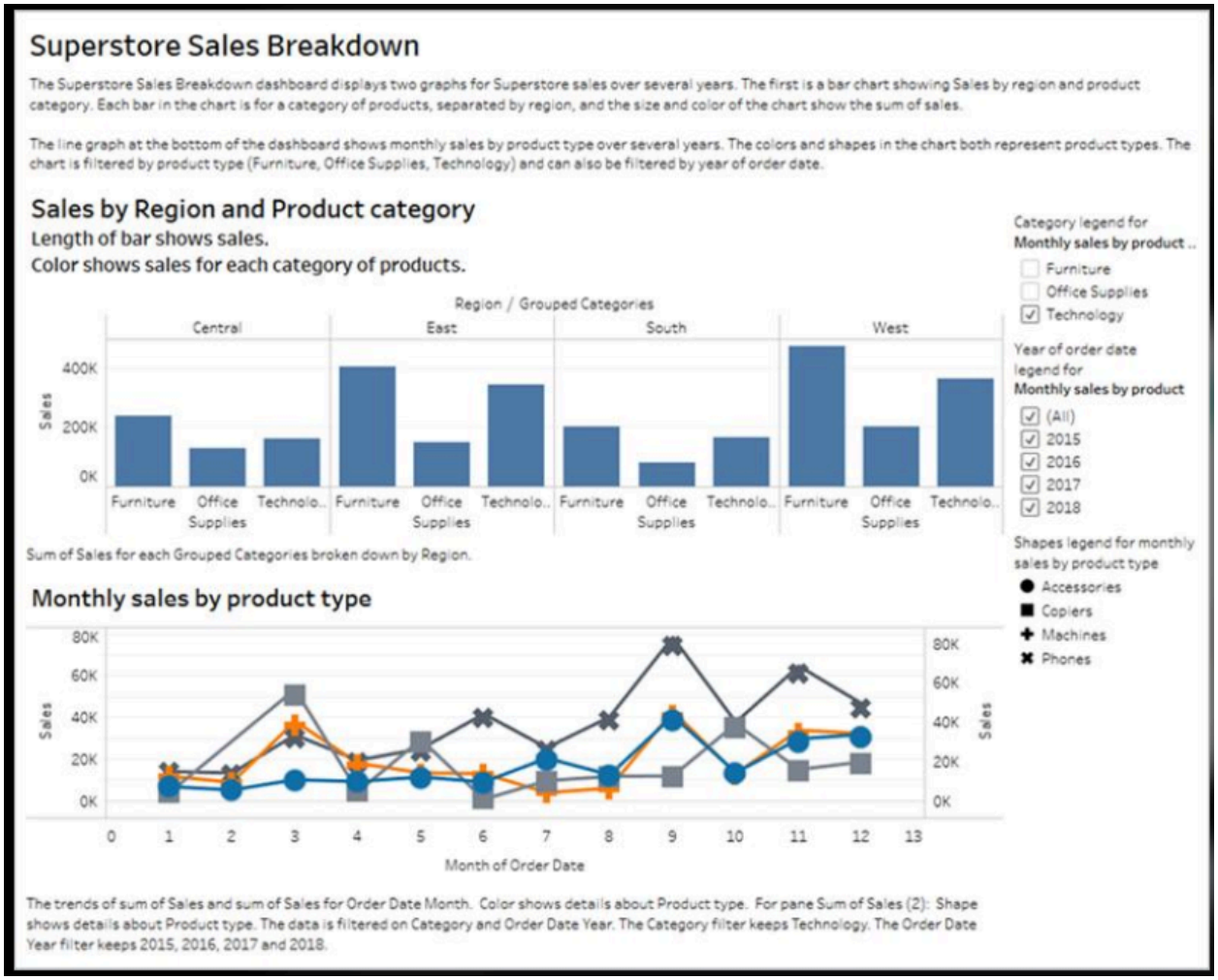
A scatter plot displaying sales and profit for each sub-category, helping identify profitable and loss-making categories.

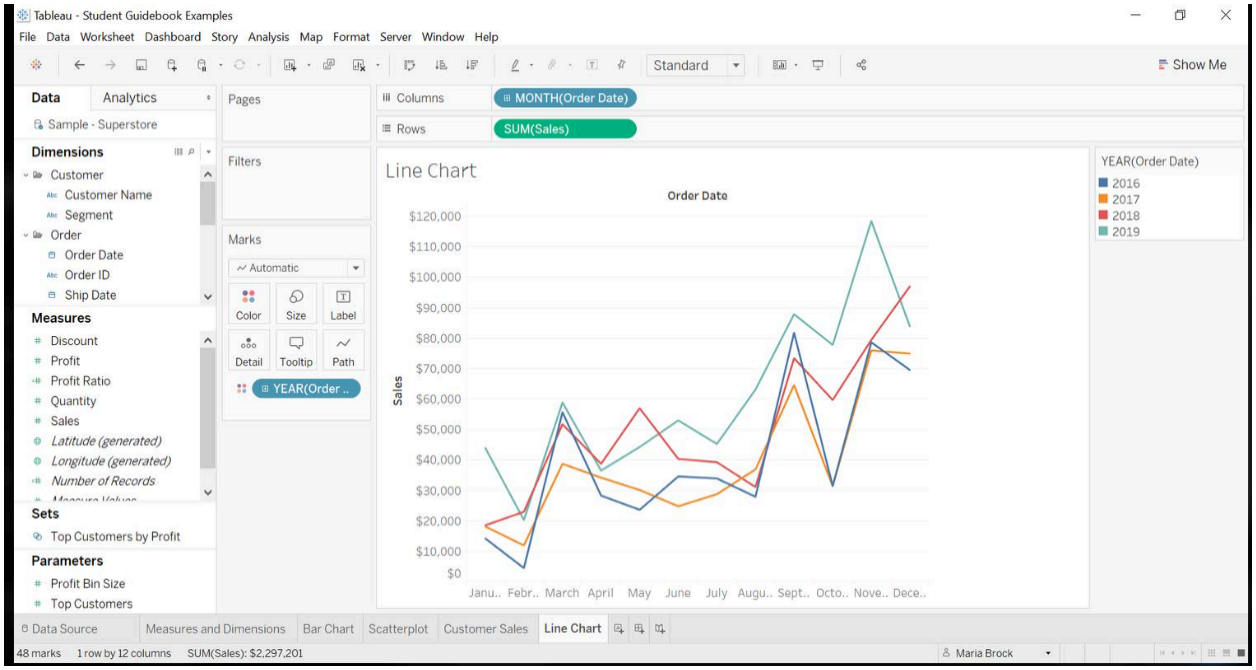
Output 4 – Interactive Sales and Profit Dashboard

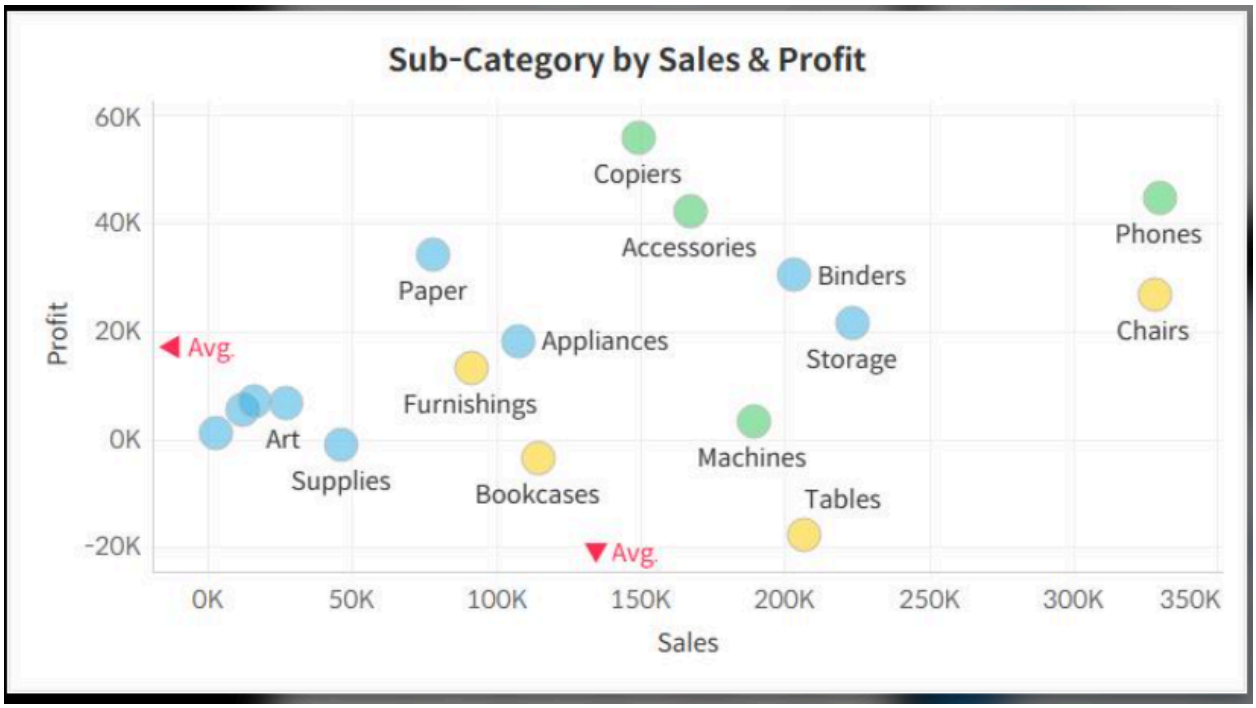
A dashboard containing all three charts, interactive filters, and dashboard actions. Selecting a category or region allows users to explore related sales and profit trends.

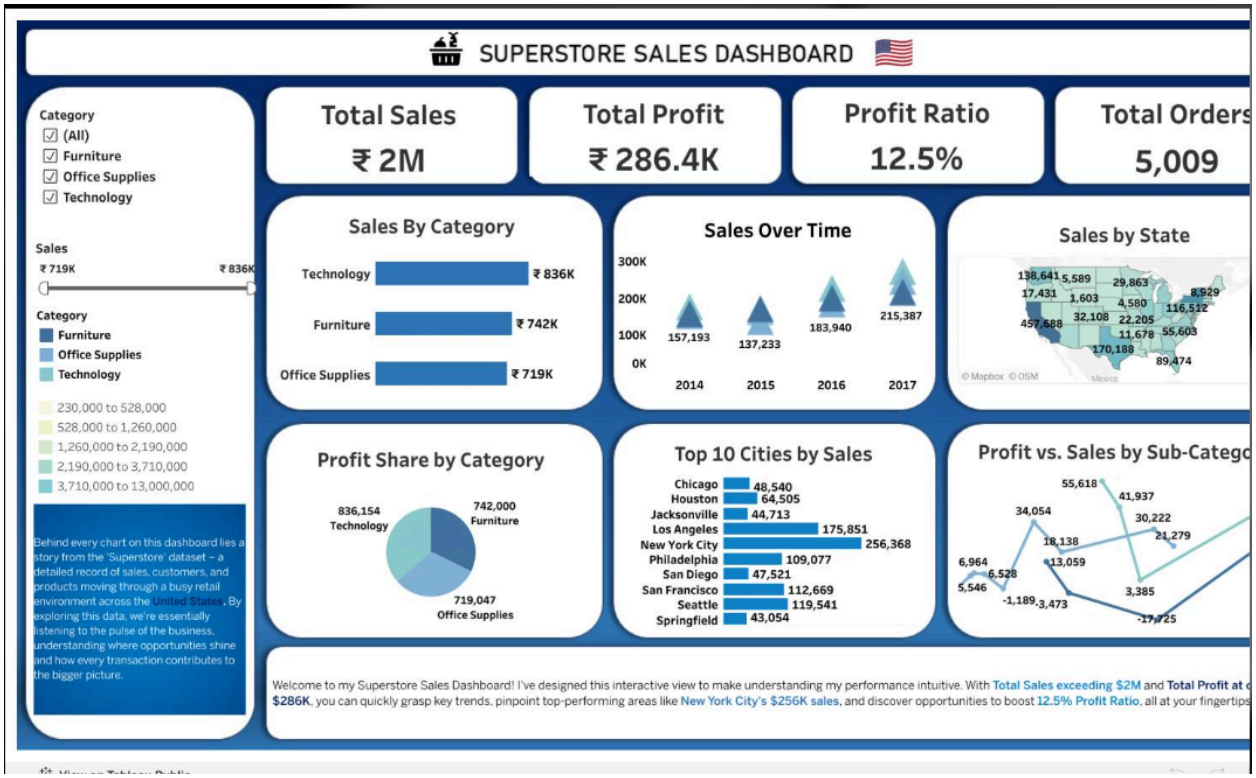
RESULT

Thus, an interactive dashboard was created in Tableau using the Sample Superstore dataset. A bar chart, line chart, and scatter plot were developed to analyse category-wise sales, sales trends over time, and the relationship between sales and profit. Interactive filters and dashboard actions were configured to enable dynamic data exploration. The dashboard was then published or exported for reporting and presentation.




Tableau - Student Guidebook Examples
File Data Worksheet Dashboard Story Analysis Map Format Server Window Help
Standard
Show Me
Data
Analytics
Sample - Superstore
Dimensions
Customer
Customer Name
Segment
Order
Order Date
Order ID
Ship Date
Measures
Discount
Profit
Profit Ratio
Quantity
Sales
Latitude (generated)
Longitude (generated)
Number of Records
Sets
Top Customers by Profit
Parameters
Profit Bin Size
Top Customers
Pages
Filters
Marks
Automatic
Color
Size
Label
Detail
Tooltip
Path
YEAR(Order ..
Columns
MONTH(Order Date)
Rows
SUM(Sales)
Line Chart
Order Date
Sales
$120,000
$110,000
$100,000
$90,000
$80,000
$70,000
$60,000
$50,000
$40,000
$30,000
$20,000
$10,000
$0
Janu.. Febr.. March April May June July Augu.. Sept.. Octo.. Nove.. Dece..
YEAR(Order Date)
2016
2017
2018
2019
Data Source
Measures and Dimensions
Bar Chart
Scatterplot
Customer Sales
Line Chart
48 marks
1 row by 12 columns
SUM(Sales): $2,297,201
Maria Brock


Sub-Category by Sales & Profit
Profit
60K
40K
20K
0K
-20K
Avg.
Avg.
Copiers
Accessories
Paper
Binders
Phones
Chairs
Storage
Appliances
Furnishings
Art
Supplies
Machines
Bookcases
Tables
0K
50K
100K
150K
200K
250K
300K
350K
Sales


SUPERSTORE SALES DASHBOARD
Category
(All)
Furniture
Office Supplies
Technology
Sales
₹ 719K
₹ 836K
Category
Furniture
Office Supplies
Technology
230,000 to 528,000
528,000 to 1,260,000
1,260,000 to 2,190,000
2,190,000 to 3,710,000
3,710,000 to 13,000,000
Behind every chart on this dashboard lies a story from the 'Superstore' dataset – a detailed record of sales, customers, and products moving through a busy retail environment across the United States. By exploring this data, we're essentially listening to the pulse of the business, understanding where opportunities shine and how every transaction contributes to the bigger picture.
Total Sales
₹ 2M
Total Profit
₹ 286.4K
Profit Ratio
12.5%
Total Orders
5,009
Sales By Category
Technology
₹ 836K
Furniture
₹ 742K
Office Supplies
₹ 719K
Sales Over Time
300K
200K
100K
0K
157,193
137,233
183,940
215,387
2014
2015
2016
2017
Sales by State
138,641
5,589
29,863
17,431
1,603
8,929
4,580
116,512
457,688
32,108
22,205
11,678
55,603
170,188
89,474
© Mapbox © OSM
Mexico
Profit Share by Category
836,154
Technology
742,000
Furniture
719,047
Office Supplies
Top 10 Cities by Sales
Chicago 48,540
Houston 64,505
Jacksonville 44,713
Los Angeles 175,851
New York City 256,368
Philadelphia 109,077
San Diego 47,521
San Francisco 112,669
Seattle 119,541
Springfield 43,054
Profit vs. Sales by Sub-Catego
55,618
41,937
34,054
30,222
21,279
18,138
13,059
6,964
6,528
5,546
-1,189
-3,473
3,385
-17,725
Welcome to my Superstore Sales Dashboard! I've designed this interactive view to make understanding my performance intuitive. With Total Sales exceeding $2M and Total Profit at
$286K, you can quickly grasp key trends, pinpoint top-performing areas like New York City's $256K sales, and discover opportunities to boost 12.5% Profit Ratio, all at your fingertips